

SOLIDWORKS API (.NET VBA C#) — Examples (48/50): Set Viewports Example_CSharp Switch_Edit_Context Example_VBNET

SOLIDWORKS API Documentation (converted from CHM)

Set Viewports Example (C#)

This example shows how to set viewports and check and change whether they're linked.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\introtosw\pressure_plate.sldprt.
// 2. Set some breakpoints (e.g., before and after panning the viewports).
// 3. Open the Immediate window.
// 4. Press F5 at each breakpoint.
//
// Postconditions:
// 1. Sets viewport arrangement to Two View-Horizontal (* Top viewport is
//    panned; bottom viewport also panned because it is linked to top
//    viewport).
// 3. Unlinks viewports (Top viewport is panned; bottom view isn't panned
//    because viewports were unlinked).
// 4. Sets viewport arrangement to a single viewport.
// 5. Examine the Immediate window.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace ViewportModelViewManager_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModelViewManager = default(ModelViewManager);
            ModelView swModelView = default(ModelView);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            // Set the viewport arrangement to Two View-Horizontal
            Debug.Print("Set viewport arrangement to Two View-Horizontal...");
            swModelViewManager = swModel.ModelViewManager;
            swModelViewManager.ViewportDisplay = (int)swViewportDisplay_e.swViewportTwoViewHorizontal;
            // Check to see if the viewports are linked
            Debug.Print("Are viewports linked? " + swModelViewManager.LinkedViews);
            // Pan the model in the top viewport
            // The model in the bottom viewport also pans
            // because it's linked to the top viewport
            Debug.Print("Panning the model in top viewport; model in bottom viewport also pans because the viewports are
linked...");
            swModelView = (ModelView)swModel.ActiveView;
            swModelView.TranslateBy(-0.0005535897435897, 0);
            swModelView.TranslateBy(-0.0005535897435897, 0);
            swModelView.TranslateBy(-0.0005535897435897, 0.0005535897435897);
            swModelView.TranslateBy(-0.003875128205128, 0.001660769230769);
            swModelView.TranslateBy(-0.004982307692308, 0.0005535897435897);
            swModelView.TranslateBy(-0.005535897435897, 0.0005535897435897);
            swModelView.TranslateBy(-0.007196666666667, 0);
            swModelView.TranslateBy(-0.006089487179487, 0);
            swModelView.TranslateBy(-0.006643076923077, 0);
            swModelView.TranslateBy(-0.004982307692308, 0);
            swModelView.TranslateBy(-0.005535897435897, 0);
            swModelView.TranslateBy(-0.002214358974359, 0);
            swModelView.TranslateBy(-0.0005535897435897, 0);
            swModelView.TranslateBy(0, -0.0005535897435897);
            swModelView.TranslateBy(0, -0.001107179487179);
        }
    }
}
```

```

swModelView.TranslateBy(0.001107179487179, -0.002214358974359);
swModelView.TranslateBy(0.001660769230769, -0.004982307692308);
swModelView.TranslateBy(0.002214358974359, -0.006643076923077);
swModelView.TranslateBy(0.001660769230769, -0.006089487179487);
swModelView.TranslateBy(0.001107179487179, -0.003321538461538);
swModelView.TranslateBy(0, -0.001107179487179);
swModelView.TranslateBy(0, 0.0005535897435897);
swModelView.TranslateBy(0, 0.001107179487179);
swModelView.TranslateBy(0.0005535897435897, 0.0005535897435897);
swModelView.TranslateBy(0.001107179487179, 0.003875128205128);
swModelView.TranslateBy(0.002767948717949, 0.004428717948718);
swModelView.TranslateBy(0.002214358974359, 0.006643076923077);
swModelView.TranslateBy(0.002767948717949, 0.005535897435897);
swModelView.TranslateBy(0.003875128205128, 0.006643076923077);
swModelView.TranslateBy(0.002214358974359, 0.004428717948718);
swModelView.TranslateBy(0, 0.001107179487179);
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(-0.001660769230769, 0);
swModelView.TranslateBy(-0.003875128205128, 0.0005535897435897);
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949);
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949);
swModelView.TranslateBy(-0.0160541025641, 0.003321538461538);
swModelView.TranslateBy(-0.01273256410256, 0.003321538461538);
swModelView.TranslateBy(-0.006643076923077, 0.001660769230769);
swModelView.TranslateBy(-0.002767948717949, 0.0005535897435897);
swModelView.TranslateBy(0.0005535897435897, 0);
swModelView.TranslateBy(0.002767948717949, 0);
swModelView.TranslateBy(0.008857435897436, -0.0005535897435897);
swModelView.TranslateBy(0.02435794871795, -0.001660769230769);
swModelView.TranslateBy(0.03044743589744, -0.002214358974359);
swModelView.TranslateBy(0.01826846153846, -0.004428717948718);
swModelView.TranslateBy(0.01217897435897, -0.004982307692308);
swModelView.TranslateBy(0.004428717948718, -0.004982307692308);
swModelView.TranslateBy(0.001660769230769, -0.003875128205128);
swModelView.TranslateBy(0, -0.001660769230769);
swModelView.TranslateBy(0.0005535897435897, -0.0005535897435897);
swModelView.TranslateBy(0.001660769230769, 0);
swModelView.TranslateBy(0.002214358974359, 0);
swModelView.TranslateBy(0.008303846153846, 0.001107179487179);
swModelView.TranslateBy(0.009411025641026, 0.004428717948718);
swModelView.TranslateBy(0.01328615384615, 0.008303846153846);
swModelView.TranslateBy(0.003875128205128, 0.007750256410256);
swModelView.TranslateBy(-0.002767948717949, 0.007750256410256);
swModelView.TranslateBy(-0.006643076923077, 0.003875128205128);
swModelView.TranslateBy(-0.006643076923077, 0);
swModelView.TranslateBy(-0.006089487179487, 0);
swModelView.TranslateBy(-0.008857435897436, 0);
swModelView.TranslateBy(-0.006643076923077, -0.001107179487179);
swModelView.TranslateBy(-0.002767948717949, -0.003321538461538);
swModelView.TranslateBy(-0.001660769230769, -0.001660769230769);
swModelView.TranslateBy(-0.0005535897435897, -0.001107179487179);
swModelView.TranslateBy(0, -0.0005535897435897);
swModelView.TranslateBy(0, 0.0005535897435897);
swModelView.TranslateBy(0, 0.0005535897435897);
// Fit the model in each viewport
swModel.ViewZoomtofit2();
// Unlink the viewports
Debug.Print("Unlink the viewports...");

```

```
swModelViewManager.LinkedViews = false;
Debug.Print("Are viewports linked? " + swModelViewManager.LinkedViews);
// Pan the model in the top viewport
// The model in the bottom viewport doesn't pan
// because it's no longer linked to the top viewport
Debug.Print("Panning the model in top viewport; model in bottom viewport doesn't pan because the viewports are no
longer linked...");
swModelView = (ModelView)swModel.ActiveView;
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(-0.0005535897435897, 0.0005535897435897);
swModelView.TranslateBy(-0.003875128205128, 0.001660769230769);
swModelView.TranslateBy(-0.004982307692308, 0.0005535897435897);
swModelView.TranslateBy(-0.005535897435897, 0.0005535897435897);
swModelView.TranslateBy(-0.007196666666667, 0);
swModelView.TranslateBy(-0.006089487179487, 0);
swModelView.TranslateBy(-0.006643076923077, 0);
swModelView.TranslateBy(-0.004982307692308, 0);
swModelView.TranslateBy(-0.005535897435897, 0);
swModelView.TranslateBy(-0.002214358974359, 0);
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(0, -0.0005535897435897);
swModelView.TranslateBy(0, -0.001107179487179);
swModelView.TranslateBy(0.001107179487179, -0.002214358974359);
swModelView.TranslateBy(0.001660769230769, -0.004982307692308);
swModelView.TranslateBy(0.002214358974359, -0.006643076923077);
swModelView.TranslateBy(0.001660769230769, -0.006089487179487);
swModelView.TranslateBy(0.001107179487179, -0.003321538461538);
swModelView.TranslateBy(0, -0.001107179487179);
swModelView.TranslateBy(0, 0.0005535897435897);
swModelView.TranslateBy(0, 0.001107179487179);
swModelView.TranslateBy(0.0005535897435897, 0.0005535897435897);
swModelView.TranslateBy(0.001107179487179, 0.003875128205128);
swModelView.TranslateBy(0.002767948717949, 0.004428717948718);
swModelView.TranslateBy(0.002214358974359, 0.006643076923077);
swModelView.TranslateBy(0.002767948717949, 0.005535897435897);
swModelView.TranslateBy(0.003875128205128, 0.006643076923077);
swModelView.TranslateBy(0.002214358974359, 0.004428717948718);
swModelView.TranslateBy(0, 0.001107179487179);
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(-0.0005535897435897, 0);
swModelView.TranslateBy(-0.001660769230769, 0);
swModelView.TranslateBy(-0.003875128205128, 0.0005535897435897);
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949);
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949);
swModelView.TranslateBy(-0.0160541025641, 0.003321538461538);
swModelView.TranslateBy(-0.01273256410256, 0.003321538461538);
swModelView.TranslateBy(-0.006643076923077, 0.001660769230769);
swModelView.TranslateBy(-0.002767948717949, 0.0005535897435897);
swModelView.TranslateBy(0.0005535897435897, 0);
swModelView.TranslateBy(0.002767948717949, 0);
swModelView.TranslateBy(0.008857435897436, -0.0005535897435897);
swModelView.TranslateBy(0.02435794871795, -0.001660769230769);
swModelView.TranslateBy(0.03044743589744, -0.002214358974359);
swModelView.TranslateBy(0.01826846153846, -0.004428717948718);
swModelView.TranslateBy(0.01217897435897, -0.004982307692308);
swModelView.TranslateBy(0.004428717948718, -0.004982307692308);
swModelView.TranslateBy(0.001660769230769, -0.003875128205128);
swModelView.TranslateBy(0, -0.001660769230769);
swModelView.TranslateBy(0.0005535897435897, -0.0005535897435897);
```

```

swModelView.TranslateBy(0.001660769230769, 0);
swModelView.TranslateBy(0.002214358974359, 0);
swModelView.TranslateBy(0.008303846153846, 0.001107179487179);
swModelView.TranslateBy(0.009411025641026, 0.004428717948718);
swModelView.TranslateBy(0.01328615384615, 0.008303846153846);
swModelView.TranslateBy(0.003875128205128, 0.007750256410256);
swModelView.TranslateBy(-0.002767948717949, 0.007750256410256);
swModelView.TranslateBy(-0.006643076923077, 0.003875128205128);
swModelView.TranslateBy(-0.006643076923077, 0);
swModelView.TranslateBy(-0.006089487179487, 0);
swModelView.TranslateBy(-0.008857435897436, 0);
swModelView.TranslateBy(-0.006643076923077, -0.001107179487179);
swModelView.TranslateBy(-0.002767948717949, -0.003321538461538);
swModelView.TranslateBy(-0.001660769230769, -0.001660769230769);
swModelView.TranslateBy(-0.0005535897435897, -0.001107179487179);
swModelView.TranslateBy(0, -0.0005535897435897);
swModelView.TranslateBy(0, 0.0005535897435897);
swModelView.TranslateBy(0, 0.0005535897435897);
// Change the viewport arrangement back to a single viewport
swModelViewManager.ViewportDisplay = (int)swViewportDisplay_e.swViewportSingle;
// Fit the model in the viewport
swModel.ViewZoomtofit2();
}
public SldWorks swApp;
}
}

```

Set Viewports Example (VBA)

This example shows how to set viewports and check and change whether they're linked.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\introtosw\pressure_plate.sldprt.
' 2. Set some breakpoints (e.g., before and after panning the viewports).
' 3. Open the Immediate window.
' 4. Press F5 at each breakpoint.
'
' Postconditions:
' 1. Sets viewport arrangement to Two View-Horizontal (* Top viewport is
'   panned; bottom viewport also panned because it is linked to top
'   viewport).
' 3. Unlinks viewports (Top viewport is panned; bottom view isn't panned
'   because viewports were unlinked).
' 4. Sets viewport arrangement to a single viewport.
' 5. Examine the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelViewManager As SldWorks.ModelViewManager
Dim swModelView As SldWorks.ModelView

```

Sub main()

```

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Set the viewport arrangement to Two View-Horizontal
    Debug.Print "Set viewport arrangement to Two View-Horizontal..."
    Set swModelViewManager = swModel.ModelViewManager
    swModelViewManager.ViewportDisplay = swViewportDisplay_e.swViewportTwoViewHorizontal

    ' Check to see if the viewports are linked
    Debug.Print "Are viewports linked? " & swModelViewManager.LinkedViews

    ' Pan the model in the top viewport
    ' The model in the bottom viewport also pans
    ' because it's linked to the top viewport
    Debug.Print "Panning the model in top viewport; model in bottom viewport also pans because the viewports are
    linked..."

    Set swModelView = swModel.ActiveView
    swModelView.TranslateBy -5.535897435897E-04, 0
    swModelView.TranslateBy -5.535897435897E-04, 0
    swModelView.TranslateBy -5.535897435897E-04, 5.535897435897E-04
    swModelView.TranslateBy -0.003875128205128, 0.001660769230769
    swModelView.TranslateBy -0.004982307692308, 5.535897435897E-04
    swModelView.TranslateBy -0.005535897435897, 5.535897435897E-04
    swModelView.TranslateBy -0.007196666666667, 0
    swModelView.TranslateBy -0.006089487179487, 0
    swModelView.TranslateBy -0.006643076923077, 0
    swModelView.TranslateBy -0.004982307692308, 0
    swModelView.TranslateBy -0.005535897435897, 0
    swModelView.TranslateBy -0.002214358974359, 0
    swModelView.TranslateBy -5.535897435897E-04, 0
    swModelView.TranslateBy 0, -5.535897435897E-04
    swModelView.TranslateBy 0, -0.001107179487179

```

swModelView.TranslateBy 0.001107179487179, -0.002214358974359
swModelView.TranslateBy 0.001660769230769, -0.004982307692308
swModelView.TranslateBy 0.002214358974359, -0.006643076923077
swModelView.TranslateBy 0.001660769230769, -0.006089487179487
swModelView.TranslateBy 0.001107179487179, -0.003321538461538
swModelView.TranslateBy 0, -0.001107179487179
swModelView.TranslateBy 0, 5.535897435897E-04
swModelView.TranslateBy 0, 0.001107179487179
swModelView.TranslateBy 5.535897435897E-04, 5.535897435897E-04
swModelView.TranslateBy 0.001107179487179, 0.003875128205128
swModelView.TranslateBy 0.002767948717949, 0.004428717948718
swModelView.TranslateBy 0.002214358974359, 0.006643076923077
swModelView.TranslateBy 0.002767948717949, 0.005535897435897
swModelView.TranslateBy 0.003875128205128, 0.006643076923077
swModelView.TranslateBy 0.002214358974359, 0.004428717948718
swModelView.TranslateBy 0, 0.001107179487179
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy -0.001660769230769, 0
swModelView.TranslateBy -0.003875128205128, 5.535897435897E-04
swModelView.TranslateBy -0.01328615384615, 0.002767948717949
swModelView.TranslateBy -0.01328615384615, 0.002767948717949
swModelView.TranslateBy -0.0160541025641, 0.003321538461538
swModelView.TranslateBy -0.01273256410256, 0.003321538461538
swModelView.TranslateBy -0.006643076923077, 0.001660769230769
swModelView.TranslateBy -0.002767948717949, 5.535897435897E-04
swModelView.TranslateBy 5.535897435897E-04, 0
swModelView.TranslateBy 0.002767948717949, 0
swModelView.TranslateBy 0.008857435897436, -5.535897435897E-04
swModelView.TranslateBy 0.02435794871795, -0.001660769230769
swModelView.TranslateBy 0.03044743589744, -0.002214358974359
swModelView.TranslateBy 0.01826846153846, -0.004428717948718
swModelView.TranslateBy 0.01217897435897, -0.004982307692308
swModelView.TranslateBy 0.004428717948718, -0.004982307692308
swModelView.TranslateBy 0.001660769230769, -0.003875128205128
swModelView.TranslateBy 0, -0.001660769230769
swModelView.TranslateBy 5.535897435897E-04, -5.535897435897E-04
swModelView.TranslateBy 0.001660769230769, 0
swModelView.TranslateBy 0.002214358974359, 0
swModelView.TranslateBy 0.008303846153846, 0.001107179487179
swModelView.TranslateBy 0.009411025641026, 0.004428717948718
swModelView.TranslateBy 0.01328615384615, 0.008303846153846
swModelView.TranslateBy 0.003875128205128, 0.007750256410256
swModelView.TranslateBy -0.002767948717949, 0.007750256410256
swModelView.TranslateBy -0.006643076923077, 0.003875128205128
swModelView.TranslateBy -0.006643076923077, 0
swModelView.TranslateBy -0.006089487179487, 0
swModelView.TranslateBy -0.008857435897436, 0
swModelView.TranslateBy -0.006643076923077, -0.001107179487179
swModelView.TranslateBy -0.002767948717949, -0.003321538461538
swModelView.TranslateBy -0.001660769230769, -0.001660769230769
swModelView.TranslateBy -5.535897435897E-04, -0.001107179487179
swModelView.TranslateBy 0, -5.535897435897E-04
swModelView.TranslateBy 0, 5.535897435897E-04
swModelView.TranslateBy 0, 5.535897435897E-04

' Fit the model in each viewport

swModel.ViewZoomtofit2

```

' Unlink the viewports
Debug.Print "Unlink the viewports..."
swModelViewManager.LinkedViews = False
Debug.Print "Are viewports linked? " & swModelViewManager.LinkedViews

' Pan the model in the top viewport
' The model in the bottom viewport doesn't pan
' because it's no longer linked to the top viewport
Debug.Print "Panning the model in top viewport; model in bottom viewport doesn't pan because the viewports are no
longer linked..."
Set swModelView = swModel.ActiveView
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy -5.535897435897E-04, 5.535897435897E-04
swModelView.TranslateBy -0.003875128205128, 0.001660769230769
swModelView.TranslateBy -0.004982307692308, 5.535897435897E-04
swModelView.TranslateBy -0.005535897435897, 5.535897435897E-04
swModelView.TranslateBy -0.007196666666667, 0
swModelView.TranslateBy -0.006089487179487, 0
swModelView.TranslateBy -0.006643076923077, 0
swModelView.TranslateBy -0.004982307692308, 0
swModelView.TranslateBy -0.005535897435897, 0
swModelView.TranslateBy -0.002214358974359, 0
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy 0, -5.535897435897E-04
swModelView.TranslateBy 0, -0.001107179487179
swModelView.TranslateBy 0.001107179487179, -0.002214358974359
swModelView.TranslateBy 0.001660769230769, -0.004982307692308
swModelView.TranslateBy 0.002214358974359, -0.006643076923077
swModelView.TranslateBy 0.001660769230769, -0.006089487179487
swModelView.TranslateBy 0.001107179487179, -0.003321538461538
swModelView.TranslateBy 0, -0.001107179487179
swModelView.TranslateBy 0, 5.535897435897E-04
swModelView.TranslateBy 0, 0.001107179487179
swModelView.TranslateBy 5.535897435897E-04, 5.535897435897E-04
swModelView.TranslateBy 0.001107179487179, 0.003875128205128
swModelView.TranslateBy 0.002767948717949, 0.004428717948718
swModelView.TranslateBy 0.002214358974359, 0.006643076923077
swModelView.TranslateBy 0.002767948717949, 0.005535897435897
swModelView.TranslateBy 0.003875128205128, 0.006643076923077
swModelView.TranslateBy 0.002214358974359, 0.004428717948718
swModelView.TranslateBy 0, 0.001107179487179
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy -5.535897435897E-04, 0
swModelView.TranslateBy -0.001660769230769, 0
swModelView.TranslateBy -0.003875128205128, 5.535897435897E-04
swModelView.TranslateBy -0.01328615384615, 0.002767948717949
swModelView.TranslateBy -0.01328615384615, 0.002767948717949
swModelView.TranslateBy -0.0160541025641, 0.003321538461538
swModelView.TranslateBy -0.01273256410256, 0.003321538461538
swModelView.TranslateBy -0.006643076923077, 0.001660769230769
swModelView.TranslateBy -0.002767948717949, 5.535897435897E-04
swModelView.TranslateBy 5.535897435897E-04, 0
swModelView.TranslateBy 0.002767948717949, 0
swModelView.TranslateBy 0.008857435897436, -5.535897435897E-04
swModelView.TranslateBy 0.02435794871795, -0.001660769230769
swModelView.TranslateBy 0.03044743589744, -0.002214358974359
swModelView.TranslateBy 0.01826846153846, -0.004428717948718
swModelView.TranslateBy 0.01217897435897, -0.004982307692308
swModelView.TranslateBy 0.004428717948718, -0.004982307692308

```

```
swModelView.TranslateBy 0.001660769230769, -0.003875128205128
swModelView.TranslateBy 0, -0.001660769230769
swModelView.TranslateBy 5.535897435897E-04, -5.535897435897E-04
swModelView.TranslateBy 0.001660769230769, 0
swModelView.TranslateBy 0.002214358974359, 0
swModelView.TranslateBy 0.008303846153846, 0.001107179487179
swModelView.TranslateBy 0.009411025641026, 0.004428717948718
swModelView.TranslateBy 0.01328615384615, 0.008303846153846
swModelView.TranslateBy 0.003875128205128, 0.007750256410256
swModelView.TranslateBy -0.002767948717949, 0.007750256410256
swModelView.TranslateBy -0.006643076923077, 0.003875128205128
swModelView.TranslateBy -0.006643076923077, 0
swModelView.TranslateBy -0.006089487179487, 0
swModelView.TranslateBy -0.008857435897436, 0
swModelView.TranslateBy -0.006643076923077, -0.001107179487179
swModelView.TranslateBy -0.002767948717949, -0.003321538461538
swModelView.TranslateBy -0.001660769230769, -0.001660769230769
swModelView.TranslateBy -5.535897435897E-04, -0.001107179487179
swModelView.TranslateBy 0, -5.535897435897E-04
swModelView.TranslateBy 0, 5.535897435897E-04
swModelView.TranslateBy 0, 5.535897435897E-04
```

```
' Change the viewport arrangement back to a single viewport
swModelViewManager.ViewportDisplay = swViewportDisplay_e.swViewportSingle

' Fit the model in the viewport
swModel.ViewZoomtofit2
```

End Sub

Set Viewports Example (VB.NET)

This example shows how to set viewports and check and change whether they're linked.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\introtosw\pressure_plate.sldprt.
' 2. Set some breakpoints (e.g., before and after panning the viewports).
' 3. Open the Immediate window.
' 4. Press F5 at each breakpoint.
'
' Postconditions:
' 1. Sets viewport arrangement to Two View-Horizontal (* Top viewport is
'   panned; bottom viewport also panned because it is linked to top
'   viewport).
' 3. Unlinks viewports (Top viewport is panned; bottom view isn't panned
'   because viewports were unlinked).
' 4. Sets viewport arrangement to a single viewport.
' 5. Examine the Immediate window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelViewManager As ModelViewManager
        Dim swModelView As ModelView

        swModel = swApp.ActiveDoc

        ' Set the viewport arrangement to Two View-Horizontal
        Debug.Print("Set viewport arrangement to Two View-Horizontal...")
        swModelViewManager = swModel.ModelViewManager
        swModelViewManager.ViewportDisplay = swViewportDisplay_e.swViewportTwoViewHorizontal

        ' Check to see if the viewports are linked
        Debug.Print("Are viewports linked? " & swModelViewManager.LinkedViews)

        ' Pan the model in the top viewport
        ' The model in the bottom viewport also pans
        ' because it's linked to the top viewport
        Debug.Print("Panning the model in top viewport; model in bottom viewport also pans because the viewports are
linked...")
        swModelView = swModel.ActiveView
        swModelView.TranslateBy(-0.0005535897435897, 0)
        swModelView.TranslateBy(-0.0005535897435897, 0)
        swModelView.TranslateBy(-0.0005535897435897, 0.0005535897435897)
        swModelView.TranslateBy(-0.003875128205128, 0.001660769230769)
        swModelView.TranslateBy(-0.004982307692308, 0.0005535897435897)
        swModelView.TranslateBy(-0.005535897435897, 0.0005535897435897)
        swModelView.TranslateBy(-0.007196666666667, 0)
        swModelView.TranslateBy(-0.006089487179487, 0)
        swModelView.TranslateBy(-0.006643076923077, 0)
        swModelView.TranslateBy(-0.004982307692308, 0)
        swModelView.TranslateBy(-0.005535897435897, 0)
        swModelView.TranslateBy(-0.002214358974359, 0)
    End Sub

```

swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(0, -0.0005535897435897)
swModelView.TranslateBy(0, -0.001107179487179)
swModelView.TranslateBy(0.001107179487179, -0.002214358974359)
swModelView.TranslateBy(0.001660769230769, -0.004982307692308)
swModelView.TranslateBy(0.002214358974359, -0.006643076923077)
swModelView.TranslateBy(0.001660769230769, -0.006089487179487)
swModelView.TranslateBy(0.001107179487179, -0.003321538461538)
swModelView.TranslateBy(0, -0.001107179487179)
swModelView.TranslateBy(0, 0.0005535897435897)
swModelView.TranslateBy(0, 0.001107179487179)
swModelView.TranslateBy(0.0005535897435897, 0.0005535897435897)
swModelView.TranslateBy(0.001107179487179, 0.003875128205128)
swModelView.TranslateBy(0.002767948717949, 0.004428717948718)
swModelView.TranslateBy(0.002214358974359, 0.006643076923077)
swModelView.TranslateBy(0.002767948717949, 0.005535897435897)
swModelView.TranslateBy(0.003875128205128, 0.006643076923077)
swModelView.TranslateBy(0.002214358974359, 0.004428717948718)
swModelView.TranslateBy(0, 0.001107179487179)
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(-0.001660769230769, 0)
swModelView.TranslateBy(-0.003875128205128, 0.0005535897435897)
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949)
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949)
swModelView.TranslateBy(-0.0160541025641, 0.003321538461538)
swModelView.TranslateBy(-0.01273256410256, 0.003321538461538)
swModelView.TranslateBy(-0.006643076923077, 0.001660769230769)
swModelView.TranslateBy(-0.002767948717949, 0.0005535897435897)
swModelView.TranslateBy(0.0005535897435897, 0)
swModelView.TranslateBy(0.002767948717949, 0)
swModelView.TranslateBy(0.008857435897436, -0.0005535897435897)
swModelView.TranslateBy(0.02435794871795, -0.001660769230769)
swModelView.TranslateBy(0.03044743589744, -0.002214358974359)
swModelView.TranslateBy(0.01826846153846, -0.004428717948718)
swModelView.TranslateBy(0.01217897435897, -0.004982307692308)
swModelView.TranslateBy(0.004428717948718, -0.004982307692308)
swModelView.TranslateBy(0.001660769230769, -0.003875128205128)
swModelView.TranslateBy(0, -0.001660769230769)
swModelView.TranslateBy(0.0005535897435897, -0.0005535897435897)
swModelView.TranslateBy(0.001660769230769, 0)
swModelView.TranslateBy(0.002214358974359, 0)
swModelView.TranslateBy(0.008303846153846, 0.001107179487179)
swModelView.TranslateBy(0.009411025641026, 0.004428717948718)
swModelView.TranslateBy(0.01328615384615, 0.008303846153846)
swModelView.TranslateBy(0.003875128205128, 0.007750256410256)
swModelView.TranslateBy(-0.002767948717949, 0.007750256410256)
swModelView.TranslateBy(-0.006643076923077, 0.003875128205128)
swModelView.TranslateBy(-0.006643076923077, 0)
swModelView.TranslateBy(-0.006089487179487, 0)
swModelView.TranslateBy(-0.008857435897436, 0)
swModelView.TranslateBy(-0.006643076923077, -0.001107179487179)
swModelView.TranslateBy(-0.002767948717949, -0.003321538461538)
swModelView.TranslateBy(-0.001660769230769, -0.001660769230769)
swModelView.TranslateBy(-0.0005535897435897, -0.001107179487179)
swModelView.TranslateBy(0, -0.0005535897435897)
swModelView.TranslateBy(0, 0.0005535897435897)
swModelView.TranslateBy(0, 0.0005535897435897)

```

' Fit the model in each viewport
swModel.ViewZoomtofit2()

' Unlink the viewports
Debug.Print("Unlink the viewports...")
swModelViewManager.LinkedViews = False
Debug.Print("Are viewports linked? " & swModelViewManager.LinkedViews)

' Pan the model in the top viewport
' The model in the bottom viewport doesn't pan
' because it's no longer linked to the top viewport
Debug.Print("Panning the model in top viewport; model in bottom viewport doesn't pan because the viewports are no
longer linked...")
swModelView = swModel.ActiveView
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(-0.0005535897435897, 0.0005535897435897)
swModelView.TranslateBy(-0.003875128205128, 0.001660769230769)
swModelView.TranslateBy(-0.004982307692308, 0.0005535897435897)
swModelView.TranslateBy(-0.005535897435897, 0.0005535897435897)
swModelView.TranslateBy(-0.007196666666667, 0)
swModelView.TranslateBy(-0.006089487179487, 0)
swModelView.TranslateBy(-0.006643076923077, 0)
swModelView.TranslateBy(-0.004982307692308, 0)
swModelView.TranslateBy(-0.005535897435897, 0)
swModelView.TranslateBy(-0.002214358974359, 0)
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(0, -0.0005535897435897)
swModelView.TranslateBy(0, -0.001107179487179)
swModelView.TranslateBy(0.001107179487179, -0.002214358974359)
swModelView.TranslateBy(0.001660769230769, -0.004982307692308)
swModelView.TranslateBy(0.002214358974359, -0.006643076923077)
swModelView.TranslateBy(0.001660769230769, -0.006089487179487)
swModelView.TranslateBy(0.001107179487179, -0.003321538461538)
swModelView.TranslateBy(0, -0.001107179487179)
swModelView.TranslateBy(0, 0.0005535897435897)
swModelView.TranslateBy(0, 0.001107179487179)
swModelView.TranslateBy(0.0005535897435897, 0.0005535897435897)
swModelView.TranslateBy(0.001107179487179, 0.003875128205128)
swModelView.TranslateBy(0.002767948717949, 0.004428717948718)
swModelView.TranslateBy(0.002214358974359, 0.006643076923077)
swModelView.TranslateBy(0.002767948717949, 0.005535897435897)
swModelView.TranslateBy(0.003875128205128, 0.006643076923077)
swModelView.TranslateBy(0.002214358974359, 0.004428717948718)
swModelView.TranslateBy(0, 0.001107179487179)
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(-0.0005535897435897, 0)
swModelView.TranslateBy(-0.001660769230769, 0)
swModelView.TranslateBy(-0.003875128205128, 0.0005535897435897)
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949)
swModelView.TranslateBy(-0.01328615384615, 0.002767948717949)
swModelView.TranslateBy(-0.0160541025641, 0.003321538461538)
swModelView.TranslateBy(-0.01273256410256, 0.003321538461538)
swModelView.TranslateBy(-0.006643076923077, 0.001660769230769)
swModelView.TranslateBy(-0.002767948717949, 0.0005535897435897)
swModelView.TranslateBy(0.0005535897435897, 0)
swModelView.TranslateBy(0.002767948717949, 0)
swModelView.TranslateBy(0.008857435897436, -0.0005535897435897)
swModelView.TranslateBy(0.02435794871795, -0.001660769230769)
swModelView.TranslateBy(0.03044743589744, -0.002214358974359)

```

```

swModelView.TranslateBy(0.01826846153846, -0.004428717948718)
swModelView.TranslateBy(0.01217897435897, -0.004982307692308)
swModelView.TranslateBy(0.004428717948718, -0.004982307692308)
swModelView.TranslateBy(0.001660769230769, -0.003875128205128)
swModelView.TranslateBy(0, -0.001660769230769)
swModelView.TranslateBy(0.0005535897435897, -0.0005535897435897)
swModelView.TranslateBy(0.001660769230769, 0)
swModelView.TranslateBy(0.002214358974359, 0)
swModelView.TranslateBy(0.008303846153846, 0.001107179487179)
swModelView.TranslateBy(0.009411025641026, 0.004428717948718)
swModelView.TranslateBy(0.01328615384615, 0.008303846153846)
swModelView.TranslateBy(0.003875128205128, 0.007750256410256)
swModelView.TranslateBy(-0.002767948717949, 0.007750256410256)
swModelView.TranslateBy(-0.006643076923077, 0.003875128205128)
swModelView.TranslateBy(-0.006643076923077, 0)
swModelView.TranslateBy(-0.006089487179487, 0)
swModelView.TranslateBy(-0.008857435897436, 0)
swModelView.TranslateBy(-0.006643076923077, -0.001107179487179)
swModelView.TranslateBy(-0.002767948717949, -0.003321538461538)
swModelView.TranslateBy(-0.001660769230769, -0.001660769230769)
swModelView.TranslateBy(-0.0005535897435897, -0.001107179487179)
swModelView.TranslateBy(0, -0.0005535897435897)
swModelView.TranslateBy(0, 0.0005535897435897)
swModelView.TranslateBy(0, 0.0005535897435897)

```

```

' Change the viewport arrangement back to a single viewport
swModelViewManager.ViewportDisplay = swViewportDisplay_e.swViewportSingle

```

```

' Fit the model in the viewport
swModel.ViewZoomtofit2()

```

```

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

```

End Class

```

Set Visible Bounding Box for Zoom to Fit Example (C#)

This example shows how to set the visible bounding box for Zoom to Fit.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open
// exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document and zooms to fit the
// the assembly in the graphics area.
// 2. When Done! is printed to the Immediate window,
// click Zoom to Fit > arm2 at the top of the
// FeatureManager design tree.
// 3. Examine the graphics area and observe the new bounding box.
//
// NOTE: Because the assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            MathPoint swMathPoint1 = default(MathPoint);
            MathPoint swMathPoint2 = default(MathPoint);
            MathUtility swMathUtility = default(MathUtility);
            int options = 0;
            double[] arr1 = null;
            double[] arr2 = null;
            double[] box = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            swModel = (ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\arm2.sldasm", (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors,
ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("arm2.SLDASM", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swModel.ViewZoomToFit2();
            swAssembly = (AssemblyDoc)swModel;
            swModel.ClearSelection2(true);
            options = (int)swBoundingBoxOptions_e.swBoundingBoxIncludeRefPlanes + (int)swBoundingBoxOptions_e.swBoundingBoxIncludeSketches;
            box = (double[])swAssembly.GetBox(options);

            //Change values
            box[0] = box[0] + box[0];
            box[1] = box[1] + box[1];
            box[2] = box[2] + box[2];
            box[3] = box[3] + box[3];
            box[4] = box[4] + box[4];
            box[5] = box[5] + box[5];

            //Apply the new values
            double[] ar1 = new double[3];
            double[] ar2 = new double[3];
            ar1[0] = box[0];
            ar1[1] = box[1];
            ar1[2] = box[2];
            ar2[0] = box[3];
            ar2[1] = box[4];
            ar2[2] = box[5];
            swMathUtility = (MathUtility)swApp.GetMathUtility();
            swMathPoint1 = (MathPoint)swMathUtility.CreatePoint(ar1);
            swMathPoint2 = (MathPoint)swMathUtility.CreatePoint(ar2);

            //Set the bounding box
            swModelDocExt.SetVisibleBox(swMathPoint1, swMathPoint2);

            //Remove the bounding box
            //swModelDocExt.RemoveVisibleBox

            Debug.Print("Done!");
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Set Visible Bounding Box for Zoom to Fit Example (VBA)

This example shows how to set the visible bounding box for Zoom to Fit.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document and zooms to fit
'    the assembly in the graphics area.
' 2. When Done! is printed to the Immediate window,
'    click Zoom to Fit > arm2 at the top of the
'    FeatureManager design tree.
' 3. Examine the graphics area and observe the new bounding box.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim swMathPoint1 As SldWorks.MathPoint
Dim swMathPoint2 As SldWorks.MathPoint
Dim swMathUtlity As SldWorks.MathUtility
Dim options As Long
Dim arr1() As Double
Dim arr2() As Double
Dim box() As Double
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
```

Sub main()

```
    Set swApp = Application.SldWorks

    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\arm2.sldasm", 2, 0, "", errors,
warnings)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("arm2.SLDASM", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ViewZoomtofit2
    Set swAssembly = swModel

    swModel.ClearSelection2 True

    options = swBoundingBoxIncludeRefPlanes + swBoundingBoxIncludeSketches
    box = swAssembly.GetBox(options)

    'Change values
    box(0) = box(0) + box(0)
    box(1) = box(1) + box(1)
    box(2) = box(2) + box(2)
    box(3) = box(3) + box(3)
    box(4) = box(4) + box(4)
    box(5) = box(5) + box(5)

    'Apply the new values
    Dim ar1(2) As Double
    Dim ar2(2) As Double

    ar1(0) = box(0)
    ar1(1) = box(1)
    ar1(2) = box(2)
    ar2(0) = box(3)
    ar2(1) = box(4)
    ar2(2) = box(5)

    Set swMathUtlity = swApp.GetMathUtility
    Set swMathPoint1 = swMathUtlity.CreatePoint(ar1)
    Set swMathPoint2 = swMathUtlity.CreatePoint(ar2)

    'Set the bounding box
    swModelDocExt.SetVisibleBox swMathPoint1, swMathPoint2

    'Remove the bounding box
    swModelDocExt.RemoveVisibleBox

    Debug.Print "Done!"
```

End Sub

Set Visible Bounding Box for Zoom to Fit Example (VB.NET)

This example shows how to set the visible bounding box for Zoom to Fit.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open
'    exists.
' 2. Open the Immediate window.
' Postconditions:
' 1. Opens the specified assembly document and zooms to fit
'    the assembly in the graphics area.
' 2. When Done! is printed to the Immediate window,
'    click Zoom to Fit > arm2 at the top of the
'    FeatureManager design tree.
' 3. Examine the graphics area and observe the new bounding box.
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim swMathPoint1 As MathPoint
        Dim swMathPoint2 As MathPoint
        Dim swMathUtility As MathUtility
        Dim options As Integer
        Dim arr1() As Double
        Dim arr2() As Double
        Dim box() As Double
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\arm2.sldasm", 2, 0, "", errors, warnings)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("arm2.SLDASM", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ViewZoomtofit2()
        swAssembly = swModel
        swModel.ClearSelection2(True)
        options = swBoundingBoxOptions_e.swBoundingBoxIncludeRefPlanes + swBoundingBoxOptions_e.swBoundingBoxIncludeSketches
        box = swAssembly.GetBox(options)

        'Change values
        box(0) = box(0) + box(0)
        box(1) = box(1) + box(1)
        box(2) = box(2) + box(2)
        box(3) = box(3) + box(3)
        box(4) = box(4) + box(4)
        box(5) = box(5) + box(5)

        'Apply the new values
        Dim ar1(2) As Double
        Dim ar2(2) As Double
        ar1(0) = box(0)
        ar1(1) = box(1)
        ar1(2) = box(2)
        ar2(0) = box(3)
        ar2(1) = box(4)
        ar2(2) = box(5)
        swMathUtility = swApp.GetMathUtility
        swMathPoint1 = swMathUtility.CreatePoint(ar1)
        swMathPoint2 = swMathUtility.CreatePoint(ar2)

        'Set the bounding box
        swModelDocExt.SetVisibleBox(swMathPoint1, swMathPoint2)

        'Remove the bounding box
        swModelDocExt.RemoveVisibleBox

        Debug.Print("Done!")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Show All Components Example (VBA)

This example shows how to show all of the components in an assembly.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Hide one or more components (right-click the component
'    and click the Hide Components button)
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Shows all hidden components.
' 2. Examine the graphics area, FeatureManager design tree,
'    and the Immediate window.
'-----
Option Explicit

Sub SetCompVisib(swComp As SldWorks.Component2)

    Dim vChildArray As Variant
    Dim swChildComp As SldWorks.Component2
    Dim i As Long

    ' Root component has no name; it is an empty string
    Debug.Print swComp.Name2 & "; [" & swComp.Visible & "]" (0 hidden; 1 visible); # of children --> " & swComp.IGetChildrenCount

    swComp.Visible = swComponentVisible

    vChildArray = swComp.GetChildren
    For i = 0 To UBound(vChildArray)
        Set swChildComp = vChildArray(i)
        SetCompVisib swChildComp
    Next i

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swAssy As SldWorks.AssemblyDoc
    Dim swConfig As SldWorks.Configuration
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swRootComp As SldWorks.Component2

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)

    SetCompVisib swRootComp

End Sub
```

Show_Bubble_ToolTip_Example_VB (Topic)

SOLIDWORKS API Help

Show Bubble ToolTip Example (VBA)

This example shows how to display a Bubble ToolTip.

```
'-----  
'  
' Preconditions: HTML file exists at the specified location.  
'  
' Postconditions: Contents of the HTML file are displayed in a Bubble ToolTip.  
'  
'-----  
  
Option Explicit  
Public Enum swArrowPosition_e  
    swArrowLeftTop = 0  
    swArrowLeftBottom = 1  
    swArrowRightTop = 2  
    swArrowRightBottom = 3  
    swArrowUpTopLeft = 4  
    swArrowUpTopRight = 5  
    swArrowDownBottomLeft = 6  
    swArrowDownBottomRight = 7  
    swArrowLeftOrRightTop = 8  
    swArrowLeftOrRightBottom = 9  
    swArrowLeftOrRight = 10  
    swArrowUpOrDownLeft = 11  
    swArrowUpOrDownRight = 12  
    swArrowUpOrDown = 13  
    swArrowNone = 14  
    swArrowUnknown = 15  
End Enum  
Sub main()  
    Const sURLpath As String = "D:/Samples/Sample_QuickTips.html"  
  
    Dim pSldWorks As Object  
    Set pSldWorks = CreateObject("SwHtmlControl.SwHtmlInterface")  
    pSldWorks.ShowBubbleToolTipAt 300, 400, swArrowLeftTop, "Sample Bubble ToolTip", "Message of Sample Bubble ToolTip", sURLpath  
End Sub  
'-----
```

Show Bubble ToolTip for PropertyManager Page Control Example (C#)

This example shows how to create a PropertyManager page. When the check box on the PropertyManager page is selected, a bubble ToolTip is displayed. This type of ToolTip is useful for validating data typed or selected by users in controls on a PropertyManager page.

```
//-----
// Preconditions:
// 1. Copy SolidWorksMacro.cs to your project.
// 2. Add a class module called PropMgrHdlr.cs and copy
//    PropMgrHdlr.cs to that module.
// 3. Click Project > Add Reference, browse to install_dir\api\redist,
//    select SolidWorks.Interop.swpublished.dll > OK.
// 4. Verify that the Tools > Options > System Options > Stop VSTA debugger
//    on macro exit checkbox is clear.
// 5. Open a part.
//
// Postconditions:
// 1. Creates and displays a PropertyManager page.
// 2. Select Sample check box to display a bubble ToolTip.
// 3. Click outside the bubble ToolTip to close it.
//    NOTE: Clearing Sample check box does not display
//    the bubble ToolTip.
//-----
//SolidWorksMacro.cs

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace CreatePropertyManagerPage_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public ModelDoc2 swModel;
        public SelectionMgr swSelMgr;
        public PropMgrHdlr pm;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Create a new instance of the PropertyManager class
            pm = new PropMgrHdlr(swApp);
            pm.Show();
        }

        public SldWorks swApp;
    }
}

//PropMgrHdlr.cs

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System;
using System.Runtime.InteropServices;
[ComVisibleAttribute(true)]
public class PropMgrHdlr : IPropertyManagerPage2Handler6
{
    //Required for PropertyManager page controls
    //Objects required for the PropertyManager page
    PropertyManagerPage2 pm_Page;
    PropertyManagerPageGroup pm_Group;
    PropertyManagerPageSelectionbox pm_Selection;
    PropertyManagerPageControl pm_Control;

    //Each object in the page needs a unique ID
    const int GroupID = 1;
    const int SelectionID = 3;
    const int ControlID = 4;
    string bubbleToolTipTitle;
    string bubbleToolTipMessage;
    string bubbleToolTipBmpFile;
    public void Show()
    {
        pm_Page.Show();
    }
    public PropMgrHdlr(SolidWorks.Interop.sldworks.SldWorks swApp)
    {
        string PageTitle = null;
        string caption = null;
        string tip = null;
    }
}
```

```

int options = 0;
int longerrors = 0;
int controlType = 0;
int alignment = 0;

//Set the variables for the page
PageTitle = "Sample PropertyManager page";
options = (int)swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton +
(int)swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton + (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage +
(int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton;

//Create the PropertyManager page
pm_Page = (PropertyManagerPage2)swApp.CreatePropertyManagerPage(PageTitle, options, this, ref longerrors);
//Make sure that the page was created properly
if (longerrors == (int)swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay)
{
    //Begin adding the controls to the page
    //Create the group box
    caption = "Sample Group Box";
    options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded;
    pm_Group = (PropertyManagerPageGroup)pm_Page.AddGroupBox(GroupID, caption, options);
    //Create selection box
    controlType = (int)swPropertyManagerPageControlType_e.swControlType_Selectionbox;
    caption = "";
    // No caption for selection boxes
    alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
    options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
    tip = "Select an edge, face, vertex, solid body, or a component";
    pm_Selection = (PropertyManagerPageSelectionbox)pm_Group.AddControl(SelectionID, (short)controlType, caption, (short)alignment,
options, tip);
    int[] filters = new int[7];
    filters[0] = (int)swSelectType_e.swSelEDGES;
    filters[1] = (int)swSelectType_e.swSelREFEDGES;
    filters[2] = (int)swSelectType_e.swSelFACES;
    filters[3] = (int)swSelectType_e.swSelVERTICES;
    filters[4] = (int)swSelectType_e.swSelSOLIDBODIES;
    filters[5] = (int)swSelectType_e.swSelCOMPONENTS;
    filters[6] = (int)swSelectType_e.swSelCOMPSDONTOVERRIDE;
    pm_Selection.SingleEntityOnly = false;
    pm_Selection.AllowMultipleSelectOfSameEntity = true;
    pm_Selection.Height = 50;
    pm_Selection.SetSelectionFilters(filters);
    // Create check box
    controlType = (int)swPropertyManagerPageControlType_e.swControlType_Checkbox;
    caption = "Sample check box";
    alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
    options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
    tip = "Check box";
    pm_Control = (PropertyManagerPageControl)pm_Group.AddControl(ControlID, (short)controlType, caption, (short)alignment, options, tip);

    bubbleToolTipTitle = "Sample bubble ToolTip Title";
    bubbleToolTipMessage = "Sample bubble ToolTip message";
    bubbleToolTipBmpFile = "";
}
else
{
    //If the page is not created
    System.Windows.Forms.MessageBox.Show("An error occurred while attempting to create the " + "PropertyManager Page");
}
}
private void PropertyManagerPage2Handler6_OnCheckboxCheck(int Id, bool isChecked)
{
    if (isChecked)
    {
        pm_Control.ShowBubbleToolTip(bubbleToolTipTitle, bubbleToolTipMessage, bubbleToolTipBmpFile);
    }
    else
    {
    }
}
}
void IPropertyManagerPage2Handler6.OnCheckboxCheck(int Id, bool isChecked)
{
    PropertyManagerPage2Handler6_OnCheckboxCheck(Id, isChecked);
}
private void PropertyManagerPage2Handler6_AfterClose()
{
}
void IPropertyManagerPage2Handler6.AfterClose()
{
    PropertyManagerPage2Handler6_AfterClose();
}
private void PropertyManagerPage2Handler6_OnButtonPress(int Id)
{
}
void IPropertyManagerPage2Handler6.OnButtonPress(int Id)
{
    PropertyManagerPage2Handler6_OnButtonPress(Id);
}
private void PropertyManagerPage2Handler6_OnClose(int Reason)
{
}
void IPropertyManagerPage2Handler6.OnClose(int Reason)
{
    PropertyManagerPage2Handler6_OnClose(Reason);
}
private void PropertyManagerPage2Handler6_OnSelectionboxCalloutCreated(int Id)
{
}
}

```

```

void IPropertyManagerPage2Handler6.OnSelectionboxCalloutCreated(int Id)
{
    PropertyManagerPage2Handler6_OnSelectionboxCalloutCreated(Id);
}
private void PropertyManagerPage2Handler6_OnComboboxEditChanged(int Id, string Text)
{
}
void IPropertyManagerPage2Handler6.OnComboboxEditChanged(int Id, string Text)
{
    PropertyManagerPage2Handler6_OnComboboxEditChanged(Id, Text);
}
private void PropertyManagerPage2Handler6_OnComboboxSelectionChanged(int Id, int Item)
{
}
void IPropertyManagerPage2Handler6.OnComboboxSelectionChanged(int Id, int Item)
{
    PropertyManagerPage2Handler6_OnComboboxSelectionChanged(Id, Item);
}
private void PropertyManagerPage2Handler6_OnGroupCheck(int Id, bool Checked)
{
}
void IPropertyManagerPage2Handler6.OnGroupCheck(int Id, bool Checked)
{
    PropertyManagerPage2Handler6_OnGroupCheck(Id, Checked);
}
private void PropertyManagerPage2Handler6_OnGroupExpand(int Id, bool Expanded)
{
}
void IPropertyManagerPage2Handler6.OnGroupExpand(int Id, bool Expanded)
{
    PropertyManagerPage2Handler6_OnGroupExpand(Id, Expanded);
}
private bool PropertyManagerPage2Handler6_OnHelp()
{
    return true;
}
bool IPropertyManagerPage2Handler6.OnHelp()
{
    return PropertyManagerPage2Handler6_OnHelp();
}
private bool PropertyManagerPage2Handler6_OnKeystroke(int Wparam, int Message, int Lparam, int Id)
{
    return true;
}
bool IPropertyManagerPage2Handler6.OnKeystroke(int Wparam, int Message, int Lparam, int Id)
{
    return PropertyManagerPage2Handler6_OnKeystroke(Wparam, Message, Lparam, Id);
}
private void PropertyManagerPage2Handler6_OnListboxSelectionChanged(int Id, int Item)
{
}
void IPropertyManagerPage2Handler6.OnListboxSelectionChanged(int Id, int Item)
{
    PropertyManagerPage2Handler6_OnListboxSelectionChanged(Id, Item);
}
private bool PropertyManagerPage2Handler6_OnNextPage()
{
    return true;
}
bool IPropertyManagerPage2Handler6.OnNextPage()
{
    return PropertyManagerPage2Handler6_OnNextPage();
}
private void PropertyManagerPage2Handler6_OnNumberboxChanged(int Id, double Value)
{
}
void IPropertyManagerPage2Handler6.OnNumberboxChanged(int Id, double Value)
{
    PropertyManagerPage2Handler6_OnNumberboxChanged(Id, Value);
}
private void PropertyManagerPage2Handler6_OnOptionCheck(int Id)
{
}
void IPropertyManagerPage2Handler6.OnOptionCheck(int Id)
{
    PropertyManagerPage2Handler6_OnOptionCheck(Id);
}
private void PropertyManagerPage2Handler6_OnPopupMenuItem(int Id)
{
}
void IPropertyManagerPage2Handler6.OnPopupMenuItem(int Id)
{
    PropertyManagerPage2Handler6_OnPopupMenuItem(Id);
}
private void PropertyManagerPage2Handler6_OnPopupMenuItemUpdate(int Id, ref int retval)
{
}
void IPropertyManagerPage2Handler6.OnPopupMenuItemUpdate(int Id, ref int retval)
{
    PropertyManagerPage2Handler6_OnPopupMenuItemUpdate(Id, ref retval);
}
private bool PropertyManagerPage2Handler6_OnPreview()
{
    return true;
}
bool IPropertyManagerPage2Handler6.OnPreview()
{
}

```

```

        return PropertyManagerPage2Handler6_OnPreview();
    }
    private bool PropertyManagerPage2Handler6_OnPreviousPage()
    {
        return true;
    }
    bool IPropertyManagerPage2Handler6.OnPreviousPage()
    {
        return PropertyManagerPage2Handler6_OnPreviousPage();
    }
    private void PropertyManagerPage2Handler6_OnRedo()
    {
    }
    void IPropertyManagerPage2Handler6.OnRedo()
    {
        PropertyManagerPage2Handler6_OnRedo();
    }
    private void PropertyManagerPage2Handler6_OnSelectionboxCalloutDestroyed(int Id)
    {
    }
    void IPropertyManagerPage2Handler6.OnSelectionboxCalloutDestroyed(int Id)
    {
        PropertyManagerPage2Handler6_OnSelectionboxCalloutDestroyed(Id);
    }
    private void PropertyManagerPage2Handler6_OnSelectionboxFocusChanged(int Id)
    {
    }
    void IPropertyManagerPage2Handler6.OnSelectionboxFocusChanged(int Id)
    {
        PropertyManagerPage2Handler6_OnSelectionboxFocusChanged(Id);
    }
    private void PropertyManagerPage2Handler6_OnSelectionboxListChanged(int Id, int Count)
    {
    }
    void IPropertyManagerPage2Handler6.OnSelectionboxListChanged(int Id, int Count)
    {
        PropertyManagerPage2Handler6_OnSelectionboxListChanged(Id, Count);
    }
    private void PropertyManagerPage2Handler6_OnSliderPositionChanged(int Id, double Value)
    {
    }
    void IPropertyManagerPage2Handler6.OnSliderPositionChanged(int Id, double Value)
    {
        PropertyManagerPage2Handler6_OnSliderPositionChanged(Id, Value);
    }
    private void PropertyManagerPage2Handler6_OnSliderTrackingCompleted(int Id, double Value)
    {
    }
    void IPropertyManagerPage2Handler6.OnSliderTrackingCompleted(int Id, double Value)
    {
        PropertyManagerPage2Handler6_OnSliderTrackingCompleted(Id, Value);
    }
    private bool PropertyManagerPage2Handler6_OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
    {
        return true;
    }
    bool IPropertyManagerPage2Handler6.OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
    {
        return PropertyManagerPage2Handler6_OnSubmitSelection(Id, Selection, SelType, ref ItemText);
    }
    private bool PropertyManagerPage2Handler6_OnTabClicked(int Id)
    {
        return true;
    }
    bool IPropertyManagerPage2Handler6.OnTabClicked(int Id)
    {
        return PropertyManagerPage2Handler6_OnTabClicked(Id);
    }
    private void PropertyManagerPage2Handler6_OnTextboxChanged(int Id, string Text)
    {
    }
    void IPropertyManagerPage2Handler6.OnTextboxChanged(int Id, string Text)
    {
        PropertyManagerPage2Handler6_OnTextboxChanged(Id, Text);
    }
    private void PropertyManagerPage2Handler6_OnUndo()
    {
    }
    void IPropertyManagerPage2Handler6.OnUndo()
    {
        PropertyManagerPage2Handler6_OnUndo();
    }
    private void PropertyManagerPage2Handler6_OnWhatsNew()
    {
    }
    void IPropertyManagerPage2Handler6.OnWhatsNew()
    {
        PropertyManagerPage2Handler6_OnWhatsNew();
    }
    private int PropertyManagerPage2Handler6_OnActiveXControlCreated(int Id, bool Status)
    {
        return 0;
    }
    int IPropertyManagerPage2Handler6.OnActiveXControlCreated(int Id, bool Status)
    {
        return PropertyManagerPage2Handler6_OnActiveXControlCreated(Id, Status);
    }
}

```

```
private void PropertyManagerPage2Handler6_AfterActivation()
{
}
void IPropertyManagerPage2Handler6.AfterActivation()
{
    PropertyManagerPage2Handler6_AfterActivation();
}
}
```

Show Bubble ToolTip for PropertyManager Page Control Example (VBA)

This example shows how to create a PropertyManager page. When the check box on the PropertyManager page is selected, a bubble ToolTip is displayed. This type of ToolTip is useful for validating data typed or selected by users in controls on a PropertyManager page.

```
'-----
' Preconditions:
' 1. Copy Main into your project.
' 2. Insert a class module called PropMgrHdlr and
'    copy PropMgrHdlr to that module.
' 3. Click Tools > References > SOLIDWORKS <version> exposed
'    type libraries for add-in use.
' 4. Open a part document.
'
' Postconditions:
' 1. Creates and displays a PropertyManager page.
' 2. Select Sample check box to display a bubble ToolTip.
' 3. Click outside the bubble ToolTip to close it.
'    NOTE: Clearing Sample check box does not display
'    the bubble ToolTip.
'-----
'Main

Option Explicit

Public swApp As SldWorks.SldWorks
Public swModel As SldWorks.ModelDoc2
Public swSelMgr As SldWorks.SelectionMgr
Public pm As PropMgrHdlr

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    'Create a new instance of the PropertyManager class
    'Calls the Class_Initialize procedure of the class
    Set pm = New PropMgrHdlr
    pm.Show

End Sub

Back to top

'PropMgrHdlr

Option Explicit

'Required for PropertyManager page controls
Implements PropertyManagerPage2Handler6

'Objects for the PropertyManager page
Dim pm_Page As PropertyManagerPage2
Dim pm_Group As PropertyManagerPageGroup
Dim pm_Selection As PropertyManagerPageSelectionbox
Dim pm_Control As PropertyManagerPageControl

'Each object in the page needs a unique ID
Const GroupID As Integer = 1
Const SelectionID As Integer = 3
Const ControlID As Integer = 4

Dim ClickedCancel As Boolean
Dim bubbleToolTipTitle As String
Dim bubbleToolTipMessage As String
Dim bubbleToolTipBmpFile As String
Dim isChecked As Boolean

Sub Show()
    pm_Page.Show
End Sub

Private Sub Class_Initialize()
    Dim PageTitle As String
    Dim caption As String
    Dim tip As String
    Dim options As Long
    Dim longerrors As Long
    Dim controlType As Integer
    Dim alignment As Integer

    'Set the variables for the page
    PageTitle = "Sample PropertyManager page"
    options = swPropertyManager_OkayButton + swPropertyManager_CancelButton + swPropertyManagerOptions_LockedPage +
swPropertyManagerOptions_PushpinButton

    'Create the PropertyManager page
    Set pm_Page = swApp.CreatePropertyManagerPage(PageTitle, options, Me, longerrors)

    'Make sure that the page was created properly
    If longerrors = swPropertyManagerPage_Okay Then
        'Begin adding the controls to the page
```

```

'Create the group box
caption = "Sample Group Box"
options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded
Set pm_Group = pm_Page.AddGroupBox(GroupID, caption, options)

'Create selection box
controlType = swControlType_Selectionbox

caption = "" ' No captions for selection boxes
alignment = swControlAlign_Indent
options = swControlOptions_Visible + swControlOptions_Enabled
tip = "Select an edge, face, vertex, solid body, or a component"
Set pm_Selection = pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip)

Dim filters(6) As Long
filters(0) = swSelEDGES
filters(1) = swSelREFEDGES
filters(2) = swSelFACES
filters(3) = swSelVERTICES
filters(4) = swSelSOLIDBODIES
filters(5) = swSelCOMPONENTS
filters(6) = swSelCOMPSDONTTOVERRIDE

pm_Selection.SingleEntityOnly = False
pm_Selection.AllowMultipleSelectOfSameEntity = True
pm_Selection.Height = 50
pm_Selection.SetSelectionFilters filters

' Create check box
controlType = swControlType_Checkbox
caption = "Sample check box"
alignment = swControlAlign_Indent
options = swControlOptions_Visible + swControlOptions_Enabled
tip = "Check box"
Set pm_Control = pm_Group.AddControl(ControlID, controlType, caption, alignment, options, tip)
isChecked = False
pm_Control.Checked = isChecked
bubbleToolTipTitle = "Sample bubble ToolTip Title"
bubbleToolTipMessage = "Sample bubble ToolTip message"
bubbleToolTipBmpFile = ""
Else 'If the page is not created
MsgBox "An error occurred while attempting to create the " & "PropertyManager Page", vbCritical
End If

End Sub
Private Sub PropertyManagerPage2Handler6_AfterActivation()
End Sub
Private Sub PropertyManagerPage2Handler6_AfterClose()
Set pm = Nothing
End Sub
Private Function PropertyManagerPage2Handler6_OnActiveXControlCreated(ByVal Id As Long, ByVal Status As Boolean) As Long
End Function
Private Sub PropertyManagerPage2Handler6_OnButtonPress(ByVal Id As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnClose(ByVal Reason As Long)
If Reason = swPropertyManagerPageClose_Cancel Then
'Cancel button was clicked
ClickedCancel = True
ElseIf Reason = swPropertyManagerPageClose_Okay Then
'OK button was clicked
ClickedCancel = False
End If
End Sub
Private Sub PropertyManagerPage2Handler6_OnCheckboxCheck(ByVal Id As Long, ByVal isChecked As Boolean)
If isChecked Then
pm_Control.ShowBubbleToolTip bubbleToolTipTitle, bubbleToolTipMessage, bubbleToolTipBmpFile
Else
End If
End Sub
Private Sub PropertyManagerPage2Handler6_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)
End Sub
Private Sub PropertyManagerPage2Handler6_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)
End Sub
Private Sub PropertyManagerPage2Handler6_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)
End Sub
Private Function PropertyManagerPage2Handler6_OnHelp() As Boolean
End Function
Private Function PropertyManagerPage2Handler6_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long) As Boolean
End Function
Private Sub PropertyManagerPage2Handler6_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)
End Sub
Private Function PropertyManagerPage2Handler6_OnNextPage() As Boolean
End Function
Private Sub PropertyManagerPage2Handler6_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)
End Sub
Private Sub PropertyManagerPage2Handler6_OnOptionCheck(ByVal Id As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnPopupMenuItem(ByVal Id As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnPopupMenuItemUpdate(ByVal Id As Long, retval As Long)
End Sub
Private Function PropertyManagerPage2Handler6_OnPreview() As Boolean
End Function
Private Function PropertyManagerPage2Handler6_OnPreviousPage() As Boolean
End Function
Private Sub PropertyManagerPage2Handler6_OnRedo()

```

```
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxCalloutCreated(ByVal Id As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxCalloutDestroyed(ByVal Id As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxFocusChanged(ByVal Id As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)
End Sub
Private Sub PropertyManagerPage2Handler6_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
End Sub
Private Sub PropertyManagerPage2Handler6_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub
Private Function PropertyManagerPage2Handler6_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ItemText As String) As Boolean
    PropertyManagerPage2Handler6_OnSubmitSelection = True
End Function
Private Function PropertyManagerPage2Handler6_OnTabClicked(ByVal Id As Long) As Boolean
End Function
Private Sub PropertyManagerPage2Handler6_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
End Sub
Private Sub PropertyManagerPage2Handler6_OnUndo()
End Sub
Private Sub PropertyManagerPage2Handler6_OnWhatsNew()
End Sub
```

Show Bubble ToolTip for PropertyManager Page Control Example (VB.NET)

This example shows how to create a PropertyManager page. When the check box on the PropertyManager page is selected, a bubble ToolTip is displayed. This type of ToolTip is useful for validating data typed or selected by users in controls on a PropertyManager page.

```

'-----
' Preconditions:
' 1. Copy SolidWorksMacro.vb to your project.
' 2. Add a class module called PropMgrHdlr.vb and copy
'    PropMgrHdlr.vb to that module.
' 3. Click Project > Add Reference, browse to install_dir\api\redist,
'    select SolidWorks.Interop.swpublished.dll > OK.
' 4. Verify that the Tools > Options > System Options > Stop VSTA debugger
'    on macro exit checkbox is clear.
' 5. Open a part.
'
' Postconditions:
' 1. Creates and displays a PropertyManager page.
' 2. Select Sample check box to display a bubble ToolTip.
' 3. Click outside the bubble ToolTip to close it.
'    NOTE: Clearing Sample check box does not display
'    the bubble ToolTip.
'-----
'SolidWorksMacro.vb

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

```

Partial Class SolidWorksMacro

```

    Public swModel As ModelDoc2
    Public swSelMgr As SelectionMgr
    Public pm As PropMgrHdlr

```

```

    Public Sub main()

```

```

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

```

```

        'Create a new instance of the PropertyManager class
        pm = New PropMgrHdlr(swApp)
        pm.Show()
    End Sub

```

```

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
End Class

```

[Back to top](#)

'PropMgrHdlr.vb

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Runtime.InteropServices

```

<ComVisibleAttribute(True)> Public Class PropMgrHdlr

```

    'Required for PropertyManager page controls
    Implements PropertyManagerPage2Handler6

```

```

    'Objects required for the PropertyManager page
    Dim pm_Page As PropertyManagerPage2
    Dim pm_Group As PropertyManagerPageGroup
    Dim pm_Selection As PropertyManagerPageSelectionbox
    Dim pm_Control As PropertyManagerPageControl

```

```

    'Each object in the page needs a unique ID
    Const GroupID As Integer = 1
    Const SelectionID As Integer = 3
    Const ControlID As Integer = 4
    Dim ClickedCancel As Boolean
    Dim bubbleToolTipTitle As String
    Dim bubbleToolTipMessage As String
    Dim bubbleToolTipBmpFile As String
    Dim isChecked As Boolean

```

```

    Sub Show()
        pm_Page.Show()
    End Sub

```

```

    Sub New(ByVal swApp As SolidWorks.Interop.sldworks.SldWorks)
        Dim pageTitle As String
        Dim caption As String
        Dim tip As String
        Dim options As Integer
        Dim longerrors As Integer

```

```

Dim controlType As Integer
Dim alignment As Integer

'Set the variables for the page
PageTitle = "Sample PropertyManager page"
options = swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton + swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton +
swPropertyManagerPageOptions_e.swPropertyManagerOptions_LockedPage + swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton

'Create the PropertyManager page
pm_Page = swApp.CreatePropertyManagerPage(PageTitle, options, Me, longererrors)

'Make sure that the page was created properly
If longererrors = swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay Then
    'Begin adding the controls to the page
    'Create the group box
    caption = "Sample Group Box"
    options = swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded
    pm_Group = pm_Page.AddGroupBox(GroupID, caption, options)

    'Create selection box
    controlType = swPropertyManagerPageControlType_e.swControlType_Selectionbox

    caption = "" ' No caption for selection boxes
    alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
    options = swAddControlOptions_e.swControlOptions_Visible + swAddControlOptions_e.swControlOptions_Enabled
    tip = "Select an edge, face, vertex, solid body, or a component"
    pm_Selection = pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip)
    Dim filters(6) As Integer
    filters(0) = swSelectType_e.swSelEDGES
    filters(1) = swSelectType_e.swSelREFEDGES
    filters(2) = swSelectType_e.swSelFACES
    filters(3) = swSelectType_e.swSelVERTICES
    filters(4) = swSelectType_e.swSelSOLIDBODIES
    filters(5) = swSelectType_e.swSelCOMPONENTS
    filters(6) = swSelectType_e.swSelCOMPSDONT OVERRIDE
    pm_Selection.SingleEntityOnly = False
    pm_Selection.AllowMultipleSelectOfSameEntity = True
    pm_Selection.Height = 50
    pm_Selection.SetSelectionFilters(filters)

    ' Create check box
    controlType = swPropertyManagerPageControlType_e.swControlType_Checkbox
    caption = "Sample check box"
    alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
    options = swAddControlOptions_e.swControlOptions_Visible + swAddControlOptions_e.swControlOptions_Enabled
    tip = "Check box"
    pm_Control = pm_Group.AddControl(ControlID, controlType, caption, alignment, options, tip)
    isChecked = False
    pm_Control.Checked = isChecked
    bubbleToolTipTitle = "Sample bubble ToolTip Title"
    bubbleToolTipMessage = "Sample bubble ToolTip message"
    bubbleToolTipBmpFile = ""
Else 'If the page is not created
    MsgBox("An error occurred while attempting to create the " & "PropertyManager Page", vbCritical)
End If
End Sub

Private Sub PropertyManagerPage2Handler6_OnCheckboxCheck(ByVal Id As Integer, ByVal isChecked As Boolean) Implements
PropertyManagerPage2Handler6.OnCheckboxCheck
    If isChecked Then
        pm_Control.ShowBubbleToolTip(bubbleToolTipTitle, bubbleToolTipMessage, bubbleToolTipBmpFile)
    Else
        End If
End Sub

Private Sub PropertyManagerPage2Handler6_AfterClose() Implements PropertyManagerPage2Handler6.AfterClose
End Sub

Private Sub PropertyManagerPage2Handler6_OnButtonPress(ByVal Id As Integer) Implements PropertyManagerPage2Handler6.OnButtonPress
End Sub

Private Sub PropertyManagerPage2Handler6_OnClose(ByVal Reason As Integer) Implements PropertyManagerPage2Handler6.OnClose
    If Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel Then
        'Cancel button was clicked
        ClickedCancel = True
    ElseIf Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay Then
        'OK button was clicked
        ClickedCancel = False
    End If
End Sub

Private Sub PropertyManagerPage2Handler6_OnSelectionboxCalloutCreated(ByVal Id As Integer) Implements
PropertyManagerPage2Handler6.OnSelectionboxCalloutCreated
End Sub

Private Sub PropertyManagerPage2Handler6_OnComboboxEditChanged(ByVal Id As Integer, ByVal Text As String) Implements
PropertyManagerPage2Handler6.OnComboboxEditChanged
End Sub

Private Sub PropertyManagerPage2Handler6_OnComboboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements
PropertyManagerPage2Handler6.OnComboboxSelectionChanged
End Sub

Private Sub PropertyManagerPage2Handler6_OnGroupCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements
PropertyManagerPage2Handler6.OnGroupCheck
End Sub

Private Sub PropertyManagerPage2Handler6_OnGroupExpand(ByVal Id As Integer, ByVal Expanded As Boolean) Implements
PropertyManagerPage2Handler6.OnGroupExpand
End Sub

Private Function PropertyManagerPage2Handler6_OnHelp() As Boolean Implements PropertyManagerPage2Handler6.OnHelp
End Function

Private Function PropertyManagerPage2Handler6_OnKeystroke(ByVal Wparam As Integer, ByVal Message As Integer, ByVal Lparam As Integer, ByVal Id
As Integer) As Boolean Implements PropertyManagerPage2Handler6.OnKeystroke
End Function

Private Sub PropertyManagerPage2Handler6_OnListboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements

```

```

PropertyManagerPage2Handler6.OnListBoxSelectionChanged
End Sub
Private Function PropertyManagerPage2Handler6_OnNextPage() As Boolean Implements PropertyManagerPage2Handler6.OnNextPage
End Function
Private Sub PropertyManagerPage2Handler6_OnNumberboxChanged(ByVal Id As Integer, ByVal Value As Double) Implements
PropertyManagerPage2Handler6.OnNumberboxChanged
End Sub
Private Sub PropertyManagerPage2Handler6_OnOptionCheck(ByVal Id As Integer) Implements PropertyManagerPage2Handler6.OnOptionCheck
End Sub
Private Sub PropertyManagerPage2Handler6_OnPopupMenuItem(ByVal Id As Integer) Implements PropertyManagerPage2Handler6.OnPopupMenuItem
End Sub
Private Sub PropertyManagerPage2Handler6_OnPopupMenuItemUpdate(ByVal Id As Integer, ByRef retval As Integer) Implements
PropertyManagerPage2Handler6.OnPopupMenuItemUpdate
End Sub
Private Function PropertyManagerPage2Handler6_OnPreview() As Boolean Implements PropertyManagerPage2Handler6.OnPreview
End Function
Private Function PropertyManagerPage2Handler6_OnPreviousPage() As Boolean Implements PropertyManagerPage2Handler6.OnPreviousPage
End Function
Private Sub PropertyManagerPage2Handler6_OnRedo() Implements PropertyManagerPage2Handler6.OnRedo
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxCalloutDestroyed(ByVal Id As Integer) Implements
PropertyManagerPage2Handler6.OnSelectionboxCalloutDestroyed
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxFocusChanged(ByVal Id As Integer) Implements
PropertyManagerPage2Handler6.OnSelectionboxFocusChanged
End Sub
Private Sub PropertyManagerPage2Handler6_OnSelectionboxListChanged(ByVal Id As Integer, ByVal Count As Integer) Implements
PropertyManagerPage2Handler6.OnSelectionboxListChanged
End Sub
Private Sub PropertyManagerPage2Handler6_OnSliderPositionChanged(ByVal Id As Integer, ByVal Value As Double) Implements
PropertyManagerPage2Handler6.OnSliderPositionChanged
End Sub
Private Sub PropertyManagerPage2Handler6_OnSliderTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements
PropertyManagerPage2Handler6.OnSliderTrackingCompleted
End Sub
Private Function PropertyManagerPage2Handler6_OnSubmitSelection(ByVal Id As Integer, ByVal Selection As Object, ByVal SelType As Integer,
ByRef ItemText As String) As Boolean Implements PropertyManagerPage2Handler6.OnSubmitSelection
PropertyManagerPage2Handler6_OnSubmitSelection = True
End Function
Private Function PropertyManagerPage2Handler6_OnTabClicked(ByVal Id As Integer) As Boolean Implements
PropertyManagerPage2Handler6.OnTabClicked
End Function
Private Sub PropertyManagerPage2Handler6_OnTextboxChanged(ByVal Id As Integer, ByVal Text As String) Implements
PropertyManagerPage2Handler6.OnTextboxChanged
End Sub
Private Sub PropertyManagerPage2Handler6_OnUndo() Implements PropertyManagerPage2Handler6.OnUndo
End Sub
Private Sub PropertyManagerPage2Handler6_OnWhatsNew() Implements PropertyManagerPage2Handler6.OnWhatsNew
End Sub
Private Function PropertyManagerPage2Handler6_OnActiveXControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements
PropertyManagerPage2Handler6.OnActiveXControlCreated
End Function
Private Sub PropertyManagerPage2Handler6_AfterActivation() Implements PropertyManagerPage2Handler6.AfterActivation
End Sub

End Class

```

Show Components and Component Configurations Names and Descriptions Example (VBA)

This example shows how to show and hide component and configuration names and descriptions in the FeatureManager design tree.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\smartcomponents\pillow_block.sldasm.
' 2. Collapse the bearing<1> component.
' 3. Widen the FeatureManager design tree.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. At each Stop statement, read the comments and press F5
'    to continue.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim SelMgr As SldWorks.SelectionMgr
Dim swFeatMgr As SldWorks.FeatureManager

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set SelMgr = Part.SelectionManager
    Set swFeatMgr = Part.FeatureManager

    Debug.Print "Tree Display settings upon opening document"
    Debug.Print vbTab & "Show Component Names" : " & swFeatMgr.ShowComponentNames
    Debug.Print vbTab & "Show Component Descriptions" : " & swFeatMgr.ShowComponentDescriptions
    Debug.Print vbTab & "Show Component Configuration Names" : " & swFeatMgr.ShowComponentConfigurationNames
    Debug.Print vbTab & "Show Component Configuration Descriptions" : " & swFeatMgr.ShowComponentConfigurationDescriptions & vbCrLf

    Stop
    ' To verify the settings, right-click pillow_block at the
    ' the top of the FeatureManager design tree and point at Tree Display
    ' Compare the settings with the output in the Immediate window
    ' Then click anywhere in the graphics area to close the short-cut menu

    ' Examine the FeatureManager design tree to
    ' verify that the names of the components are not shown

    If (swFeatMgr.ShowComponentDescriptions) Then
        swFeatMgr.ShowComponentDescriptions = False
    Else
        swFeatMgr.ShowComponentDescriptions = True
    End If

    Stop
    ' Show Component Descriptions is set to true

    ' Examine the FeatureManager design tree, and
    ' verify that the descriptions of the components
    ' are shown

    If (swFeatMgr.ShowComponentConfigurationNames) Then
        swFeatMgr.ShowComponentConfigurationNames = False
    Else
        swFeatMgr.ShowComponentConfigurationNames = True
    End If

    Stop
    ' Show Component Configuration Names is set to false

    ' Examine the FeatureManager design tree, and
    ' verify that the names of the configurations are
    ' not shown

    If (swFeatMgr.ShowComponentConfigurationDescriptions) Then
        swFeatMgr.ShowComponentConfigurationDescriptions = False
    Else
        swFeatMgr.ShowComponentConfigurationDescriptions = True
    End If

    Stop
    ' Show Component Configuration Descriptions is set to true

    ' Examine the FeatureManager design tree, and
    ' verify that the descriptions of the configurations
    ' are shown

    Debug.Print "Display modified settings"
    Debug.Print vbTab & "Show Component Names" : " & swFeatMgr.ShowComponentNames
    Debug.Print vbTab & "Show Component Descriptions" : " & swFeatMgr.ShowComponentDescriptions
    Debug.Print vbTab & "Show Component Configuration Names" : " & swFeatMgr.ShowComponentConfigurationNames
    Debug.Print vbTab & "Show Component Configuration Descriptions" : " & swFeatMgr.ShowComponentConfigurationDescriptions & vbCrLf

    ' FeatureManager::ShowComponentNames and FeatureManager::ShowComponentDescriptions
    ' cannot both be set to false

```

```

swFeatMgr.ShowComponentDescriptions = True
If (swFeatMgr.ShowComponentNames) Then
    swFeatMgr.ShowComponentNames = False
End If

Debug.Print "Try to set FeatureManager::ShowComponentDescriptions to false while FetureManager::ShowComponentNames is false"
Debug.Print vbTab & "Before call to Show Component Descriptions : " & swFeatMgr.ShowComponentDescriptions
swFeatMgr.ShowComponentDescriptions = False ' This call does not work because FeatureManager::ShowComponentNames display is set to false
Debug.Print vbTab & "After call to Show Component Descriptions : " & swFeatMgr.ShowComponentDescriptions & vbCrLf

' Restore original settings
If (swFeatMgr.ShowComponentDescriptions) Then
    swFeatMgr.ShowComponentNames = True
    swFeatMgr.ShowComponentDescriptions = False
Else
    swFeatMgr.ShowComponentNames = True
End If

If (swFeatMgr.ShowComponentConfigurationNames) Then
    swFeatMgr.ShowComponentConfigurationNames = False
Else
    swFeatMgr.ShowComponentConfigurationNames = True
End If

If (swFeatMgr.ShowComponentConfigurationDescriptions) Then
    swFeatMgr.ShowComponentConfigurationDescriptions = False
Else
    swFeatMgr.ShowComponentConfigurationDescriptions = True
End If

Debug.Print "Original settings restored"
Debug.Print vbTab & "Show Component Names                : " & swFeatMgr.ShowComponentNames
Debug.Print vbTab & "Show Component Descriptions            : " & swFeatMgr.ShowComponentDescriptions
Debug.Print vbTab & "Show Component Configuration Names    : " & swFeatMgr.ShowComponentConfigurationNames
Debug.Print vbTab & "Show Component Configuration Descriptions : " & swFeatMgr.ShowComponentConfigurationDescriptions & vbCrLf

Debug.Print "Macro done executing!"

End Sub

```

Show Dimensions in Drawing Sheet Example (VBA)

This example shows how to show all of the dimensions in a drawing sheet whether the dimensions are hidden or visible.

NOTE: In the SOLIDWORKS Design user interface, you can hide a dimension in a drawing view using the shortcut menu. The corresponding method to do this in the SOLIDWORKS API is `IModelDoc2::HideDimension`. However, there is no ready way to show a hidden dimension in the user interface without first selecting the dimension. This example shows how to traverse all display dimensions in a drawing sheet and show them.

```
'-----
' Preconditions:
' 1. Open install\samples\tutorial\api\advdrawings\foodprocessor.sldprt.
' 2. Box-select all dimensions in DrawingView1, right-click any
'    extension line, and click Hide.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Iterates all drawing views and shows all dimensions
'    in DrawingView1.
' 2. Examine the drawing and Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Sub ProcessDrawing(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swView As SldWorks.View)
    Dim swAnn As SldWorks.Annotation

    Debug.Print " " & swView.Name
    Set swAnn = swView.GetFirstAnnotation2
    Do While Not Nothing Is swAnn
        If swDisplayDimension = swAnn.GetType Then
            Debug.Print " " & swAnn.GetName
            swAnn.Visible = swAnnotationVisible
        End If
        Set swAnn = swAnn.GetNext2
    Loop
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Debug.Print "File = " & swModel.GetPathName

    Set swView = swDraw.GetFirstView
    Do While Not Nothing Is swView
        ProcessDrawing swApp, swDraw, swView
        Set swView = swView.GetNextView
    Loop

    swModel.GraphicsRedraw2

End Sub
```

Show_Named_View_Example_VB (Topic)

SOLIDWORKS API Help

Show Named View Example (VBA)

This example shows how to get the name of a named view in an open model document.

```
'-----  
'  
' Preconditions: Model document is open.  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Sub main()  
    Dim swModel As SldWorks.ModelDoc2  
    Dim lIdx As Long  
    Dim vTransform As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    For lIdx = swStandardViews_e.swFrontView To swStandardViews_e.swDimetricView  
        Debug.Print "View = " & PrintViewName(lIdx) & " (" & lIdx & ")"  
        vTransform = swModel.GetStandardViewRotation(lIdx)  
        swModel.ShowNamedView2 "", lIdx  
        Stop  
    Next lIdx  
End Sub  
  
Private Function PrintViewName(ByVal nView As swStandardViews_e) As String  
    PrintViewName = "<not a standard view>"  
    Select Case (nView)  
        Case swStandardViews_e.swBackView  
            PrintViewName = "Back"  
        Case swStandardViews_e.swBottomView  
            PrintViewName = "Bottom"  
        Case swStandardViews_e.swDimetricView  
            PrintViewName = "Dimetric"  
        Case swStandardViews_e.swFrontView  
            PrintViewName = "Front"  
        Case swStandardViews_e.swIsometricView  
            PrintViewName = "Isometric"  
        Case swStandardViews_e.swLeftView  
            PrintViewName = "Left"  
        Case swStandardViews_e.swRightView  
            PrintViewName = "Right"  
        Case swStandardViews_e.swTopView  
            PrintViewName = "Top"  
        Case swStandardViews_e.swTrimetricView  
            PrintViewName = "Trimetric"  
    End Select  
End Function
```

Simplify Spline Example (VBA)

This example shows how to simplify a spline.

```
'-----  
' Preconditions:  
' 1. Open a part containing a sketch of a spline.  
' 2. Select the spline in the graphics area.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Simplifies the spline.  
' 2. Examine the Immediate window and graphics area.  
'-----  
Option Explicit  
  
Sub DumpSketchSplineInfo(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSkSpline As SldWorks.SketchSpline)  
  
    Dim vSplinePt As Variant  
    Dim swSkPt As SldWorks.SketchPoint  
    Dim i As Long  
  
    ' Spline passes through these points  
    vSplinePt = swSkSpline.GetPoints2  
    For i = 0 To UBound(vSplinePt)  
        Set swSkPt = vSplinePt(i)  
        Debug.Print " SketchSplinePt[" & i & "] = (" & swSkPt.X * 1000# & ", " & swSkPt.Y * 1000# & ", " & swSkPt.Z * 1000# & ") mm"  
    Next i  
End Sub  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSkSeg As SldWorks.SketchSegment  
    Dim swSkSpline As SldWorks.SketchSpline  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swSkSeg = swSelMgr.GetSelectedObject5(1)  
    Set swSkSpline = swSkSeg  
  
    Debug.Print "Before:"  
    DumpSketchSplineInfo swApp, swModel, swSkSpline  
    Debug.Print ""  
    ' Tolerance value is in meters  
    swModel.SimplifySpline 0.1  
    Debug.Print "After:"  
    DumpSketchSplineInfo swApp, swModel, swSkSpline  
    Debug.Print ""  
  
End Sub
```

Get Sketch Spline Parameters Example (C++ COM)

This example shows how to get splines parameter data from sketches and how to unpack that data.

The method ISketch::!GetSplineParams2 returns an array, so this code must be used in an in-process DLL. Otherwise, use the method ISketch::GetSplineParams2 that returns a VARIANT.

```
// Union for use unpacking the data in the first two array elements for each spline
union SplinePackedData
{
    double doublevalue[2];
    int  intvalue[4];
};

void GetSketchSplines(LP SKETCH pSketch, CSTRING* Message)
{
    // Assume that pSketch has been set to a valid sketch already

    CSTRING  Text;
    double*  pData;
    long  DataSize, NoSplines;

    // Get the spline parameters
    if(S_OK != pSketch->GetSplineParamsCount(&DataSize, &NoSplines) )return;

    if(NoSplines==0)
    {
        *Message += " Has no spline curves\r\n";
        return;
    }
    pData = (double*)malloc(sizeof(double) * DataSize);

    if(S_OK != pSketch->!GetSplineParams2(pData) )return;

    // Format the spline parameter data
    ExtractSplineData ( DataSize, pData, Message );

}

////////////////////////////////////
// Method: ExtractSplinesData
// Arguments: DataSize - Size of the array containing the splines parameters
//  pData - Data array containing splines parameters
// Return: Message - A CSTRING for returning the formatted information
// Description:Extract the splines parameter data returned
//  from GetSplineParams and format it into a
//  string for display.
////////////////////////////////////

void CGetSketchesApp::ExtractSplinesData(int DataSize, double* pData, CSTRING* Message)
{
    int  Index = 0;
    int  Dim;
    int  Order;
    int  nCtrlPoints;
    int  Periodic;
    int  nKnots;
    double  Knot;
    SplinePackedData* pPackedData;
    int  i,j;
    CSTRING  Line;

    // Continue until all the data is processed, i.e. 1 loop per spline
    while ( Index < DataSize )
    {
        // Unpack the information about the spline
        pPackedData = (SplinePackedData*)&pData[Index];
        Dim = pPackedData->intvalue[0];
```

```

Order = pPackedData->intvalue[1];
nCtrlPoints = pPackedData->intvalue[2];
Periodic = pPackedData->intvalue[3];
nKnots = Periodic ? nCtrlPoints+1 : nCtrlPoints+Order;

// Format Text
Line.Format(_T("Spline\nDimension %d\nNumControlPoints %d\n\n
Num of Knots %d\nPeriodic %d\n"),Dim, nCtrlPoints, nKnots, Periodic);

*Message += Line;

// Extract Control points
Index+=2;
// Step past packed data
for ( i=0; i<nCtrlPoints; i++ )
{
    // Assume 3 dimensions
    double CtrlPoint[3];

    for ( j=0; j<3; j++ )
    {
        CtrlPoint[j] = pData[Index++];
    }

    // Format text (in this case assume 3 dimensions)
    Line.Format(_T("Control Point: %lf %lf %lf\n"),
        CtrlPoint[0],
        CtrlPoint[1],
        CtrlPoint[2]);
    *Message += Line;
}

// Extract Knots
for ( i=0; i<nKnots; i++ )
{
    Knot = pData[Index++];

    // Format text
    Line.Format(_T("Knot: %lf\n"), Knot);
    *Message += Line;
}
}
}

```

Offset Sketch Example (C#)

This example shows how to offset a sketch.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates a new part.
// 2. Sketches a line.
// 3. Offsets the line 2.54 mm in both directions.
// 4. Examine the graphics area.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Runtime.InteropServices;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SketchManager swSketchManager;
        ModelDocExtension swModelDocExt;
        SketchSegment swSketchSegment;
        bool status;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            swSketchManager.InsertSketch(true);
            status = swModelDocExt.SelectByID("Top Plane", "PLANE", -0.0770466366627886, 0.00233041566204965, 0.0390732100788036, false, 0, null,
0);
            swModel.ClearSelection2(true);

            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.081532, 0.028203, 0.0, -0.029228, -0.017264, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.029228, -0.017264, 0.0, 0.035382, -0.025468, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.035382, -0.025468, 0.0, 0.087008, -0.070346, 0.0);
            swModel.ClearSelection2(true);

            status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0, 0, 0, false, 1, null, 0);
            status = swSketchManager.SketchOffset2(0.00254, true, true, (int)swSkOffsetCapEndType_e.swSkOffsetArcCaps, (int)swSkOffsetMakeConstruct
ionType_e.swSkOffsetMakeBothConstruction, true);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
        }

        public SldWorks swApp;
    }
}
```

Offset Sketch Example (VBA)

This example shows how to offset a sketch.

```

'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Creates a new part.
' 2. Sketches a line.
' 3. Offsets the line 2.54 mm in both directions.
' 4. Examine the graphics area.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchManager As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks

    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prt", 0, 0, 0)
    Set swModel = swApp.ActiveDoc
    Set swSketchManager = swModel.SketchManager
    Set swModelDocExt = swModel.Extension

    swSketchManager.InsertSketch True
    status = swModelDocExt.SelectByID2("Top Plane", "PLANE", -7.70466366627886E-02, 2.33041566204965E-03, 3.90732100788036E-02, False, 0, Nothing,
0)
    swModel.ClearSelection2 True

    Set swSketchSegment = swSketchManager.CreateLine(-0.081532, 0.028203, 0#, -0.029228, -0.017264, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(-0.029228, -0.017264, 0#, 0.035382, -0.025468, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.035382, -0.025468, 0#, 0.087008, -0.070346, 0#)
    swModel.ClearSelection2 True

    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, False, 1, Nothing, 0)
    status = swSketchManager.SketchOffset2(0.00254, True, True, swSketchOffsetCapEndType_e.swSketchOffsetArcCaps,
swSketchOffsetMakeConstructionType_e.swSketchOffsetMakeBothConstruction, True)

    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

End Sub

```

Offset Sketch Example (VB.NET)

This example shows how to offset a sketch.

```

'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Creates a new part.
' 2. Sketches a line.
' 3. Offsets the line 2.54 mm in both directions.
' 4. Examine the graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSketchManager As SketchManager
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchSegment As SketchSegment
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swModel = swApp.ActiveDoc
        swSketchManager = swModel.SketchManager
        swModelDocExt = swModel.Extension
        swSketchManager.InsertSketch(True)
        status = swModelDocExt.SelectByID("Top Plane", "PLANE", -0.077046636627886, 0.00233041566204965, 0.0390732100788036, False, 0, Nothing, 0
    )

        swModel.ClearSelection2(True)

        swSketchSegment = swSketchManager.CreateLine(-0.081532, 0.028203, 0.0#, -0.029228, -0.017264, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(-0.029228, -0.017264, 0.0#, 0.035382, -0.025468, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.035382, -0.025468, 0.0#, 0.087008, -0.070346, 0.0#)
        swModel.ClearSelection2(True)

        status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0, 0, 0, False, 1, Nothing, 0)
        status = swSketchManager.SketchOffset2(0.00254, True, True, swSkOffsetCapEndType_e.swSkOffsetArcCaps, swSkOffsetMakeConstructionType_e.swSkOffsetMakeBothConstruction, True)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Sketch Spline Example (VBA)

This example shows how to sketch a spline.

```
'-----
' Preconditions: Open a new part.
'
' Postconditions:
' 1. Creates a sketch.
' 2. Sketches a spline.
' 3. Examine the graphics area.
'-----
```

Option Explicit

```
Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim nPtData(35) As Double
    Dim vPtData As Variant
    Dim swSketchSeg As SldWorks.SketchSegment
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    nPtData(0) = 3      'Dimension
    nPtData(1) = 4      'Order
    nPtData(2) = 7      'Number of control points
    nPtData(3) = 0      'Periodicity

    ' Knots
    nPtData(4) = 0
    nPtData(5) = 0
    nPtData(6) = 0
    nPtData(7) = 0
    nPtData(8) = 0.295784969875007
    nPtData(9) = 0.295784969875007
    nPtData(10) = 0.588801176674805
    nPtData(11) = 1
    nPtData(12) = 1
    nPtData(13) = 1
    nPtData(14) = 1

    ' Control points
    nPtData(15) = -101.019623655914 / 1000#:    nPtData(16) = -9.38844086021505 / 1000#:    nPtData(17) = 0#
    nPtData(18) = -119.993564790426 / 1000#:    nPtData(19) = 13.6067814289365 / 1000#:    nPtData(20) = 0#
    nPtData(21) = -138.967505924938 / 1000#:    nPtData(22) = 36.602003718088 / 1000#:    nPtData(23) = 0#
    nPtData(24) = 15.8143947928824 / 1000#:    nPtData(25) = 19.0600549700683 / 1000#:    nPtData(26) = 0#
    nPtData(27) = -59.4979409454141 / 1000#:    nPtData(28) = -24.6206927754025 / 1000#:    nPtData(29) = 0#
    nPtData(30) = 12.2077094083999 / 1000#:    nPtData(31) = -18.3231635953553 / 1000#:    nPtData(32) = 0#
    nPtData(33) = 54.0774193548387 / 1000#:    nPtData(34) = -14.6459677419355 / 1000#:    nPtData(35) = 0#

    vPtData = nPtData

    swModel.InsertSketch2 True
    bRet = swModel.SketchSplineByEqnParams2((vPtData))
    swModel.InsertSketch2 True
End Sub
```

Skip Items in Array Example (VBA)

This example shows how to skip items in an array.

```
' -----  
  
Private Sub CommandButton1_Click()  
    Set pSwApp = GetObject("", "SldWorks.Application")  
    Set pPart = pSwApp.ActiveDoc  
    pPart.SelectByID "LPattern1", "BODYFEATURE", 0, 0, 0  
    Set pSelMan = pPart.SelectionManager  
    Set pFeature = pSelMan.GetSelectedObject5(1)  
    Set pFeatureData = pFeature.GetDefinition  
    Dim skip As Variant  
    Dim skip2(0 To 3) As Long  
    skip = pFeatureData.SkippedItemArray  
    skip2(0) = 1  
    skip2(1) = 2  
    skip2(2) = 3  
    skip2(3) = 4  
    skip = skip2  
    pFeatureData.SkippedItemArray = (skip)  
    pFeature.ModifyDefinition pFeatureData, pPart, Nothing  
    pFeatureData.ReleaseSelectionAccess  
End Sub
```

Slice a Model Example (C#)

This example shows how to slice a model.

```
// =====
// Preconditions:
// 1. Open Public_Documents\SOLIDWORKS\SOLIDWORKS 2020\samples\tutorial\api\cube.sldprt.
// 2. Change units to MKS.
// 3. Open the Immediate window.
// Postconditions:
// 1. Selects Front Plane as the first slicing plane.
// 2. Gets the default slicing parameters.
// 3. Specifies new slicing parameters.
// 4. Performs slicing and adds Slice1 to the FeatureManager design tree.
// 5. Inspect the graphics area and the Immediate window.
// NOTE: Because the model is used elsewhere, do not save changes to it.
// =====
```

```
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace SliceModel_CSharp
```

```
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SlicingData sliceData;
        private bool boolstatus;
        private int errors;
        private object vars;

        public void Main()
        {
            modDoc = (ModelDoc2)swApp.ActiveDoc;
            swFeatMgr = modDoc.FeatureManager;

            // Select Case the first slicing plane
            boolstatus = modDoc.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null/* TODO Change to
default(_) if this is not a reference type */, 0);
            // Get default slicing parameters
            sliceData = (SlicingData)swFeatMgr.GetSlicingData();

            Debug.Print("Default slicing values:");
            Debug.Print("");
            Debug.Print("Bounding box direction 1: " + sliceData.GetBoundingBoxDirection1());
            Debug.Print("Bounding box direction 2: " + sliceData.GetBoundingBoxDirection2());
            Debug.Print("Bounding box direction 3: " + sliceData.GetBoundingBoxDirection3());
            Debug.Print("Bounding box direction 4: " + sliceData.GetBoundingBoxDirection4());
            Debug.Print("Reverse direction? " + sliceData.ReverseDirection);
            Debug.Print("Number of slices: " + sliceData.NumberOfPlanes);
            Debug.Print("Distance between slices: " + sliceData.Offset);
            Debug.Print("Type of slicing as defined in swSlicingTypes_e: " + sliceData.SlicesToGenerate);
            Debug.Print("Add slicing planes and sketches to folder? " + sliceData.AddSlicingPlanesAndSketchesToFolder);

            sliceData.ReverseDirection = false;
```

```

boolstatus = sliceData.SetBoundingBoxDirection1(0.1397);
boolstatus = sliceData.SetBoundingBoxDirection2(0.1397);
boolstatus = sliceData.SetBoundingBoxDirection3(-0.1397);
boolstatus = sliceData.SetBoundingBoxDirection4(-0.1397);
sliceData.NumberOfPlanes = 6;
sliceData.Offset = 0.01;
sliceData.SlicesToGenerate = (int)swSlicingTypes_e.swSlicingTypes_Rectangle;
sliceData.AddSlicingPlanesAndSketchesToFolder = true;

Debug.Print("");
Debug.Print("New values:");
Debug.Print("");
Debug.Print("Bounding box direction 1: " + sliceData.GetBoundingBoxDirection1());
Debug.Print("Bounding box direction 2: " + sliceData.GetBoundingBoxDirection2());
Debug.Print("Bounding box direction 3: " + sliceData.GetBoundingBoxDirection3());
Debug.Print("Bounding box direction 4: " + sliceData.GetBoundingBoxDirection4());
Debug.Print("Reverse direction? " + sliceData.ReverseDirection);
Debug.Print("Number of slices: " + sliceData.NumberOfPlanes);
Debug.Print("Distance between slices: " + sliceData.Offset);
Debug.Print("Type of slicing as defined in swSlicingTypes_e: " + sliceData.SlicesToGenerate);
Debug.Print("Add slicing planes and sketches to folder? " + sliceData.AddSlicingPlanesAndSketchesToFolder);

vars = swFeatMgr.InsertSlicing(sliceData, out errors);

Debug.Print("");
Debug.Print("Slicing error code as defined in swInsertSlicingError_e: " + errors);
}

/// <summary>
///     ''' The SldWorks swApp variable is pre-assigned for you.
///     ''' </summary>
public SldWorks swApp;
}

}

```

SolidWorks.Interop.sldworks Assembly

	SOLIDWORKS API
---	----------------

 Namespaces

Namespace	Description
SolidWorks.Interop.sldworks	SOLIDWORKS API

SOLIDWORKS_Visible_or_BackGround_Example_VB (Topic)

SOLIDWORKS API Help

SOLIDWORKS Design Visible or Background Example (VBA)

These examples show several situations for running or attaching to SOLIDWORKS Design sessions through the API.

SOLIDWORKS Design not running: Launch invisibly and end SOLIDWORKS Design session

```
Sub Macro1()  
    Dim swApp As Object  
    Set swApp = CreateObject("SldWorks.Application")  
    ' Close the SOLIDWORKS Design application. If you do not perform this step, SOLIDWORKS Design  
    ' continues to run. If SOLIDWORKS Design is running in the background, the user is  
    ' unaware that SOLIDWORKS Design is running and consuming their system resources.  
    swApp.ExitApp  
  
    Set swApp = Nothing  
End Sub
```

SOLIDWORKS Design running: Attach to existing SOLIDWORKS Design session and end SOLIDWORKS Design session

```
Sub Macro2()  
    Dim swApp As Object  
    Set swApp = CreateObject("SldWorks.Application")  
    swApp.ExitApp  
    Set swApp = Nothing  
End Sub
```

SOLIDWORKS Design not running: Launch invisibly, become visible, and end SOLIDWORKS Design session

```
Sub Macro3()  
    Dim swApp As Object  
    Dim Part As Object  
    Set swApp = CreateObject("SldWorks.Application")  
    swApp.Visible = True  
    swApp.ExitApp  
    Set swApp = Nothing  
End Sub
```

SOLIDWORKS Design not running: Launch invisibly, become visible, and leave SOLIDWORKS Design running

```
Sub Macro4()  
    Dim swApp As Object  
    Set swApp = CreateObject("SldWorks.Application")  
    swApp.Visible = True  
    swApp.UserControl = True  
    ' Give control to the user, which leaves SOLIDWORKS Design running  
End Sub
```

SOLIDWORKS Design not running: Launch invisibly, create part invisibly, close part, and end SOLIDWORKS Design session

```
Sub Macro5()  
    Dim swApp As Object  
    Dim Part As Object  
    ' This must be explicitly defined for ISldWorks::ActivateDoc2  
    Dim errors As Long  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Part = swApp.NewPart  
    Set Part = swApp.ActivateDoc2("Part1", True, errors)  
    Set Part = Nothing  
    swApp.CloseDoc "Part1"  
    swApp.ExitApp  
    Set swApp = Nothing  
End Sub
```

SOLIDWORKS Design not running: Launch invisibly, create part invisibly, make the part and SOLIDWORKS Design visible, and leave SOLIDWORKS Design running

```
Sub Macro6()  
    Dim swApp As Object  
    Dim Part As Object  
    Set swApp = CreateObject("SldWorks.Application")
```

```
Set Part = swApp.NewPart  
Set Part = swApp.ActivateDoc("Part1")  
Part.Visible = True  
    ' Make the part and SOLIDWORKS Design visible  
swApp.UserControl = True  
    ' Give control to the user, which leaves SOLIDWORKS Design running  
Set Part = Nothing  
swApp.CloseDoc "Part1"  
End Sub
```

Sort Table Example (C#)

This example shows how to sort bill of materials, hole, general, and weldment cut list tables.

```
//-----
// Preconditions:
// 1. Open a drawing of an assembly with multiple components
//    and one of these tables:
//    * General
//    * Hole chart
//    * Weldment cut list
//    * Bill of materials parts-only table that contains
//      four columns: ITEM NO., PART NUMBER, DESCRIPTION, and QTY.
// 2. Click the move-table icon in the upper-left corner
//    of the table to open the table's PropertyManager page.
// 3. If the table is a BOM table, then verify that
//    Follow assembly order in Item Numbers is not selected.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Sorts:
//    * BOM table as follows:
//      * Primary sort is on column one (PART NUMBER).
//      * Secondary sort is on column three (QTY.).
//      * There is no tertiary sort.
//      * All column sorts are literal ascending.
//      * Rows are sorted into categories in the following order:
//        1. Parts
//        2. User-defined
//    * Inserts a row in the table.
//    * Attempts to sort the table again by applying the
//      saved sorting scheme.
//    * General table in descending order on the first column.
//    * Hole table in ascending order on the TAG column.
//    * Weldment cut list table in descending order on the third
//      column.
// 2. Inspect the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        IModelDoc2 swModDoc = null;
        ITableAnnotation swTable = null;

        public void Main()
        {
            GetSelTable();
            SortSelTable();
        }

        public void GetSelTable()
        {
            swModDoc = (IModelDoc2)swApp.ActiveDoc;
            ISelectionMgr swSM = null;
            swSM = (ISelectionMgr)swModDoc.SelectionManager;
            swTable = (ITableAnnotation)swSM.GetSelectedObject6(1, -1);
            swModDoc.ClearSelection2(true);
        }

        public void SortSelTable()
        {
            if (swTable != null)
            {
                swTableAnnotationType_e tableType = (swTableAnnotationType_e)swTable.Type;
                bool status = false;
                switch (tableType)
                {
                    case swTableAnnotationType_e.swTableAnnotation_General:
                        status = SortGeneralTable();
                        break;
                    case swTableAnnotationType_e.swTableAnnotation_HoleChart:
                        status = SortHoleChartTable();
                        break;
                    case swTableAnnotationType_e.swTableAnnotation_WeldmentCutList:
                        status = SortWeldmentCutlistTable();
                        break;
                    case swTableAnnotationType_e.swTableAnnotation_BillofMaterials:
                        status = SortBOMTable();
                }
            }
        }
    }
}
```

```

        break;
    default:
        Debug.Print("Unspecified table type selected.");
        break;
    }
}

public bool SortGeneralTable()
{
    Debug.Print("Table type selected : swTableAnnotation_General");
    IGeneralTableAnnotation swSpecTable = (IGeneralTableAnnotation)swTable;
    bool status = swSpecTable.Sort(0, false); //sort descending
    return status;
}

public bool SortHoleChartTable()
{
    Debug.Print("Table type selected : swTableAnnotation_HoleChart");
    IHoleTableAnnotation swSpecTable = (IHoleTableAnnotation)swTable;
    bool status = swSpecTable.Sort(0, true); //sort ascending
    return status;
}

public bool SortWeldmentCutlistTable()
{
    Debug.Print("Table type selected : swTableAnnotation_WeldmentCutList");
    IWeldmentCutListAnnotation swSpecTable = (IWeldmentCutListAnnotation)swTable;
    bool status = swSpecTable.Sort(2, false); //sort descending
    return status;
}

public bool SortBOMTable()
{
    Debug.Print("Table type selected : swTableAnnotation_BillOfMaterials");
    IBomTableAnnotation swSpecTable = (IBomTableAnnotation)swTable;
    BomTableSortData swSortData = swSpecTable.GetBomTableSortData();
    bool sortSaved = false;

    // Specify the sort order indexes for three columns in the table
    // Specify the direction of sort for the three sort order indexes
    swSortData.set_ColumnIndex(0, 1); // primary sort on column 1
    swSortData.set_Ascending(0, true); // sort ascending
    swSortData.set_ColumnIndex(1, 3); // secondary sort on column 3
    swSortData.set_Ascending(1, true); // sort ascending
    swSortData.set_ColumnIndex(2, -1); // no tertiary sort

    // Specify the literal sort method
    swSortData.SortMethod = (int)swBomTableSortMethod_e.swBomTableSortMethod_Literal;

    int pos1 = swSortData.get_ColumnIndex(0);
    Debug.Print("Column for primary sort is " + pos1); // should be 1
    int pos2 = swSortData.get_ColumnIndex(1);
    Debug.Print("Column for secondary sort is " + pos2); // should be 3
    int pos3 = swSortData.get_ColumnIndex(2);
    Debug.Print("Column for tertiary sort is " + pos3); // should be -1

    int[] listGrpArray = null;
    object grpArray = null;
    grpArray = listGrpArray;
    bool bWantGrp = true;

    if(bWantGrp)
    {
        // Sort rows into part and user-defined categories
        listGrpArray = new int[2];

        listGrpArray[0] = (int)swBomTableSortItemGroup_e.swBomTableSortItemGroup_Parts;
        listGrpArray[1] = (int)swBomTableSortItemGroup_e.swBomTableSortItemGroup_Other;
        grpArray = listGrpArray;
    }
    swSortData.ItemGroups = grpArray;

    // After sorting, do not re-number the items
    swSortData.DoNotChangeItemNumber = true;

    // Save the sort settings to the BOM Table in the drawing
    sortSaved = swSortData.SaveCurrentSortParameters;

    bool status = false;
    status = swSpecTable.Sort(swSortData);
    Debug.Print("BOM table sorted? " + status);

    // Insert a row
    swTable.InsertRow(2, 2);

    // Apply the sort settings saved in the BOM table
    swSpecTable.ApplySavedSortScheme(swSortData);
    Debug.Print("BOM sorting scheme applied? " + status);
}

```

```
        return status;
    }
    public SldWorks swApp;
}
```

Sort Table Example (VBA)

This example shows how to sort bill of materials, hole, general, and weldment cut list tables.

```
'-----
' Preconditions:
' 1. Open a drawing of an assembly with multiple components
'    and one of these tables:
'    * General
'    * Hole chart
'    * Weldment cut list
'    * Bill of materials parts-only table that contains
'      four columns: ITEM NO., PART NUMBER, DESCRIPTION, and QTY.
' 2. Click the move-table icon in the upper-left corner
'    of the table to open the table's PropertyManager page.
' 3. If the table is a BOM table, then verify that
'    Follow assembly order in Item Numbers is not selected.
' 4. Open the Immediate window.
```

```
' Postconditions:
' 1. Sorts:
'    * BOM table as follows:
'      * Primary sort is on column one (PART NUMBER).
'      * Secondary sort is on column three (QTY.).
'      * There is no tertiary sort.
'      * All column sorts are literal ascending.
'      * Rows are sorted into categories in the following order:
'        1. Parts
'        2. User-defined
'    * Inserts a row in the table.
'    * Attempts to sort the table again by applying the
'      saved sorting scheme.
'    * General table in descending order on the first column.
'    * Hole table in ascending order on the TAG column.
'    * Weldment cut list table in descending order on the third
'      column.
' 2. Inspect the Immediate window.
```

```
Dim swApp As SldWorks.SldWorks
Dim swModDoc As SldWorks.IModelDoc2
Dim swTable As SldWorks.ITableAnnotation
```

```
Option Explicit
```

```
Public Sub Main()
    Set swApp = Application.SldWorks
    GetSelTable
    SortSelTable
End Sub
```

```
Public Sub GetSelTable()
    Set swModDoc = swApp.ActiveDoc
    Dim swSM As ISelectionMgr
    Set swSM = swModDoc.SelectionManager
    Set swTable = swSM.GetSelectedObject6(1, -1)
    swModDoc.ClearSelection2 (True)
End Sub
```

```
Public Sub SortSelTable()
    Dim tableType As swTableAnnotationType_e
    tableType = swTable.Type
    Dim status As Boolean
    Select Case tableType
        Case swTableAnnotationType_e.swTableAnnotation_General
            status = SortGeneralTable()
        End Case
        Case swTableAnnotationType_e.swTableAnnotation_HoleChart
            status = SortHoleChartTable()
        End Case
        Case swTableAnnotationType_e.swTableAnnotation_WeldmentCutList
            status = SortWeldmentCutlistTable()
        End Case
        Case swTableAnnotationType_e.swTableAnnotation_BillOfMaterials
            status = SortBOMTable()
        End Case
        Case Else
            Debug.Print ("Unspecified table type selected.")
        End Case
    End Select
End Sub
```

```
Public Function SortGeneralTable() As Boolean
    Debug.Print ("Table type selected: swTableAnnotation_General")
    Dim swSpecTable As IGeneralTableAnnotation
    Set swSpecTable = swTable
```

```

    Dim status As Boolean
    status = swSpecTable.Sort(0, False) 'sort descending
End Function

Public Function SortHoleChartTable() As Boolean
    Debug.Print ("Table type selected: swTableAnnotation_HoleChart")
    Dim swSpecTable As IHoleTableAnnotation
    Set swSpecTable = swTable
    Dim status As Boolean
    status = swSpecTable.Sort(0, True) 'sort ascending
End Function

Public Function SortWeldmentCutlistTable() As Boolean
    Debug.Print ("Table type selected: swTableAnnotation_WeldmentCutList")
    Dim swSpecTable As IWeldmentCutListAnnotation
    Set swSpecTable = swTable
    Dim status As Boolean
    status = swSpecTable.Sort(2, False) 'sort descending
End Function

Public Function SortBOMTable() As Boolean
    Debug.Print ("Table type selected: swTableAnnotation_BillofMaterials")
    Dim swSpecTable As IBomTableAnnotation
    Set swSpecTable = swTable
    Dim swSortData As BomTableSortData
    Dim sortSaved As Boolean

    Set swSortData = swSpecTable.GetBomTableSortData

    ' Specify the sort order indexes for three columns in the table
    ' Specify the direction of sort for the three sort order indexes
    swSortData.ColumnIndex(0) = 1 ' primary sort on column 1
    swSortData.Ascending(0) = True ' sort ascending
    swSortData.ColumnIndex(1) = 3 ' secondary sort on column 3
    swSortData.Ascending(1) = True ' sort ascending
    swSortData.ColumnIndex(2) = -1 ' no tertiary sort

    ' Specify the literal sort method
    swSortData.SortMethod = swBomTableSortMethod_Literal

    Dim pos1 As Integer
    pos1 = swSortData.ColumnIndex(0)
    Debug.Print ("Column for primary sort is " & pos1) ' should be 1
    Dim pos2 As Integer
    pos2 = swSortData.ColumnIndex(1)
    Debug.Print ("Column for secondary sort is " & pos2) ' should be 3
    Dim pos3 As Integer
    pos3 = swSortData.ColumnIndex(2)
    Debug.Print ("Column for tertiary sort is " & pos3) ' should be -1

    Dim listGrpArray(1) As Integer
    Dim bWantGrp As Boolean
    bWantGrp = True

    If bWantGrp Then
        ' Sort rows into part and user-defined categories

        listGrpArray(0) = swBomTableSortItemGroup_Parts
        listGrpArray(1) = swBomTableSortItemGroup_Other
    End If

    swSortData.ItemGroups = listGrpArray

    ' After sorting, do not re-number the items
    swSortData.DoNotChangeItemNumber = True

    ' Save sorting scheme to the BOM table in the drawing
    sortSaved = swSortData.SaveCurrentSortParameters

    Dim status As Boolean
    status = swSpecTable.Sort(swSortData)
    Debug.Print ("BOM table sorted: " & status)

    ' Insert a row
    swTable.InsertRow 2, 2

    ' Apply sorting scheme saved in the BOM table
    status = swSpecTable.ApplySavedSortScheme(swSortData)
    Debug.Print ("BOM sorting scheme applied: " & status)
End Function

```

Sort Table Example (VB.NET)

This example shows how to sort bill of materials, hole, general, and weldment cut list tables.

```

'-----
' Preconditions:
' 1. Open a drawing of an assembly with multiple components
'    and one of these tables:
'    * General
'    * Hole chart
'    * Weldment cut list
'    * Bill of materials parts-only table that contains
'      four columns: ITEM NO., PART NUMBER, DESCRIPTION, and QTY.
' 2. Click the move-table icon in the upper-left corner
'    of the table to open the table's PropertyManager page.
' 3. If the table is a BOM table, then verify that
'    Follow assembly order in Item Numbers is not selected.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Sorts:
'    * BOM table as follows:
'      * Primary sort is on column one (PART NUMBER).
'      * Secondary sort is on column three (QTY.).
'      * There is no tertiary sort.
'      * All column sorts are literal ascending.
'      * Rows are sorted into categories in the following order:
'        1. Parts
'        2. User-defined
'      * Inserts a row in the table.
'      * Attempts to sort the table again by applying the
'        saved sorting scheme.
'    * General table in descending order on the first column.
'    * Hole table in ascending order on the TAG column.
'    * Weldment cut list table in descending order on the third
'      column.
' 2. Inspect the Immediate window.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Diagnostics
```

```
Partial Public Class SolidWorksMacro
```

```
Private swModDoc As IModelDoc2 = Nothing
```

```
Private swTable As ITableAnnotation = Nothing
```

```
Public Sub Main()
    GetSelTable()
    SortSelTable()
End Sub
```

```
Public Sub GetSelTable()
    swModDoc = DirectCast(swApp.ActiveDoc, IModelDoc2)
    Dim swSM As ISelectionMgr = Nothing
    swSM = DirectCast(swModDoc.SelectionManager, ISelectionMgr)
    swTable = DirectCast(swSM.GetSelectedObject6(1, -1), ITableAnnotation)
    swModDoc.ClearSelection2(True)
End Sub
```

```
Public Sub SortSelTable()
    If swTable IsNot Nothing Then
        Dim tableType As swTableAnnotationType_e = DirectCast(swTable.Type, swTableAnnotationType_e)
        Dim status As Boolean = False
        Select Case tableType
            Case swTableAnnotationType_e.swTableAnnotation_General
                status = SortGeneralTable()
                Exit Select
            Case swTableAnnotationType_e.swTableAnnotation_HoleChart
                status = SortHoleChartTable()
                Exit Select
            Case swTableAnnotationType_e.swTableAnnotation_WeldmentCutList
                status = SortWeldmentCutlistTable()
                Exit Select
            Case swTableAnnotationType_e.swTableAnnotation_BillOfMaterials
                status = SortBOMTable()
                Exit Select
            Case Else
                Debug.Print("Unspecified table type selected.")
                Exit Select
        End Select
    End If
End Sub
```

```

Public Function SortGeneralTable() As Boolean
    Debug.Print("Table type selected : swTableAnnotation_General")
    Dim swSpecTable As IGeneralTableAnnotation = DirectCast(swTable, IGeneralTableAnnotation)
    Dim status As Boolean = swSpecTable.Sort(0, False) 'sort descending

    Return status
End Function

Public Function SortHoleChartTable() As Boolean
    Debug.Print("Table type selected : swTableAnnotation_HoleChart")
    Dim swSpecTable As IHoleTableAnnotation = DirectCast(swTable, IHoleTableAnnotation)
    Dim status As Boolean = swSpecTable.Sort(0, True) 'sort ascending

    Return status
End Function

Public Function SortWeldmentCutlistTable() As Boolean
    Debug.Print("Table type selected : swTableAnnotation_WeldmentCutList")
    Dim swSpecTable As IWeldmentCutListAnnotation = DirectCast(swTable, IWeldmentCutListAnnotation)
    Dim status As Boolean = swSpecTable.Sort(2, False) 'sort descending

    Return status
End Function

Public Function SortBOMTable() As Boolean
    Debug.Print("Table type selected : swTableAnnotation_BillOfMaterials")
    Dim swSpecTable As IBomTableAnnotation = DirectCast(swTable, IBomTableAnnotation)
    Dim swSortData As BomTableSortData = swSpecTable.GetBomTableSortData()
    Dim sortSaved As Boolean

    ' Specify the sort order indexes for three columns in the table
    ' Specify the direction of sort for the three sort order indexes
    swSortData.ColumnIndex(0) = 1 ' primary sort on column 1
    swSortData.Ascending(0) = True ' sort ascending

    swSortData.ColumnIndex(1) = 3 ' secondary sort on column 3
    swSortData.Ascending(1) = True ' sort ascending

    swSortData.ColumnIndex(2) = -1 ' no tertiary sort

    ' Specify the literal sort method
    swSortData.SortMethod = swBomTableSortMethod_e.swBomTableSortMethod_Literal

    Dim pos1 As Integer = swSortData.ColumnIndex(0)
    Debug.Print("Column for primary sort is " & pos1) ' should be 1

    Dim pos2 As Integer = swSortData.ColumnIndex(1)
    Debug.Print("Column for secondary sort is " & pos2) ' should be 3

    Dim pos3 As Integer = swSortData.ColumnIndex(2)
    Debug.Print("Column for tertiary sort is " & pos3) ' should be -1

    Dim listGrpArray As Integer() = Nothing
    Dim grpArray As Object = Nothing
    grpArray = listGrpArray
    Dim bWantGrp As Boolean = True

    If bWantGrp Then
        ' Sort rows into part and user-defined categories
        listGrpArray = New Integer(1) {}

        listGrpArray(0) = CInt(swBomTableSortItemGroup_e.swBomTableSortItemGroup_Parts)
        listGrpArray(1) = CInt(swBomTableSortItemGroup_e.swBomTableSortItemGroup_Other)
        grpArray = listGrpArray
    End If
    swSortData.ItemGroups = grpArray

    ' After sorting, do not re-number the items
    swSortData.DoNotChangeItemNumber = True

    ' Save the sort settings to the BOM table in the drawing
    sortSaved = swSortData.SaveCurrentSortParameters

    Dim status As Boolean
    status = swSpecTable.Sort(swSortData)
    Debug.Print("BOM table sorted? " & status)

    ' Insert a row
    swTable.InsertRow(2, 2)

    ' Apply the sort settings saved in the BOM table
    swSpecTable.ApplySavedSortScheme(swSortData)
    Debug.Print("BOM sorting scheme applied? " & status)

    Return status
End Function

Public swApp As SldWorks
End Class

```

Specify IGES Levels and Values, Then Import IGES File Example (VBA)

This example shows how to:

- specify levels and values for importing IGES data.
- import an IGES file.

```

'-----
' Preconditions: Substitute the path and name
' of your IGES file where noted in the code.
'
' Postconditions:
' 1. Creates a folder named Layer 25.
' 2. Imports the specified IGES file into SOLIDWORKS Design
' and moves the imported IGES features to Layer 25.
' 3. To verify, examine the graphics area and
' FeatureManager design tree.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim model As SldWorks.ModelDoc2
    Dim boolstatus As Boolean
    Dim fileName As String
    Dim argString As String
    Dim importData As SldWorks.ImportIgesData
    Dim Err As Long
    Dim orgSetting As Boolean
    Dim allLevels As Boolean
    Dim vOnlyLev As Variant
    Dim onlyLev(0 To 1) As Long
    Dim oneLev As Long
    Dim lastFeature As SldWorks.Feature
    Dim newFolder As SldWorks.Feature
    Dim newFolderName As String
    Dim lastFeatureName As String

    Set swApp = Application.SldWorks
    Set model = swApp.ActiveDoc

    ' Substitute the path and name of your IGES file
    fileName = "path\IGES_file_name.igs"

    ' "r" means open new document
    ' "i" means insert into existing document

    If model Is Nothing Then
        argString = "r"
    Else ' There is an existing part, so use it
        argString = "i"
    End If

    ' Fill in the import data
    Set importData = swApp.GetImportFileData(fileName)
    If Not importData Is Nothing Then
        ' Test the various flags
        importData.IncludeSurfaces = True
        importData.IncludeCurves = True
        importData.CurvesAsSketches = True ' False = Curves as Curves
        importData.ProcessByLevel = False

        ' Test all levels
        ' allLevels = True
        ' False = levels specified in vOnlyLev
        ' newFolderName = "All levels"
        ' or, test multiple levels
        ' onlyLev(0) = 0
        ' onlyLev(1) = 6
        ' vOnlyLev = onlyLev
        ' newFolderName = "Layer 0 and 6"
        ' Or, test individual levels

        oneLev = 25
        vOnlyLev = oneLev
        newFolderName = "Layer " & Format(oneLev)
        boolstatus = importData.SetLevels(allLevels, (vOnlyLev))
    End If

    ' Keep the last feature to determine what's been added
    ' If this is a new document, that cannot be done, so
    ' just hard code the name of the Origin feature,
    ' which is currently the last feature in a new part document
    ' It is better to always create a new
    ' document first, and then call SldWorks::LoadFile4
    ' with "i" argString to avoid this potential problem

    If Not model Is Nothing Then
        Set lastFeature = model.FeatureByPositionReverse(0)
    End If

```

```

        lastFeatureName = lastFeature.Name
Else
    lastFeatureName = "Origin"
End If

' Setting this user preference to true means that the IGES
' dialog is displayed
' Setting this user preference to false means that the
' IGES dialog is not displayed, and the import IGES data
' is used if it is passed in, or, if it is not,
' then the default values for the dialog are used

orgSetting = swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swIGESImportShowLevel)
swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swIGESImportShowLevel, False

Set model = swApp.LoadFile4(fileName, argString, importData, Err)

' swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swIGESImportShowLevel, orgSetting

' If the SldWorks::LoadFile4 failed, do not continue
If model Is Nothing Then
    Debug.Print "Problem loading file. Error message = " & Err
    Exit Sub
End If

' Retrieve all of the features that were created
' and move them into their own new folder

model.ClearSelection2 True

' Select features that are then used by FeatureManager::InsertFeatureTreeFolder2
' Either method of selection seems to take the same amount of time

boolstatus = select_new_features_individually(model, lastFeatureName)
boolstatus = multiselect_new_features(model, lastFeatureName)

If (boolstatus) Then
    Set newFolder = model.FeatureManager.InsertFeatureTreeFolder2(swFeatureTreeFolder_Containing)
    If Not newFolder Is Nothing Then
        newFolder.Name = newFolderName
    End If
    model.ClearSelection2 True
End If
End Sub

Private Function select_new_features_individually(model As SldWorks.ModelDoc2, lastFeatureName As String) As Boolean

    Dim testFeature As SldWorks.Feature
    Dim loopCount As Long
    Dim boolstatus As Boolean

    select_new_features_individually = False

    loopCount = 0
    Set testFeature = model.FeatureByPositionReverse(loopCount)
    While (Not testFeature Is Nothing) And (Not testFeature.Name = lastFeatureName)
        loopCount = loopCount + 1
        boolstatus = testFeature.Select2(True, 0)
        If Not boolstatus = 0 Then
            select_new_features_individually = True
        End If
        Set testFeature = model.FeatureByPositionReverse(loopCount)
    Wend
End Function

Private Function multiselect_new_features(model As SldWorks.ModelDoc2, lastFeatureName As String) As Boolean

    Dim testFeature As SldWorks.Feature
    Dim loopCount As Long
    Dim featureList() As SldWorks.Feature
    Dim vFeatureList As Variant
    Dim longstatus As Long
    Dim selMgr As SldWorks.SelectionMgr
    Dim selData As SldWorks.SelectData

    multiselect_new_features = False

    loopCount = 0
    Set testFeature = model.FeatureByPositionReverse(loopCount)

    While (Not testFeature Is Nothing) And (Not testFeature.Name = lastFeatureName)
        loopCount = loopCount + 1
        Set testFeature = model.FeatureByPositionReverse(loopCount)
    Wend

    ReDim featureList(0 To loopCount - 1)

    loopCount = 0
    Set testFeature = model.FeatureByPositionReverse(loopCount)

    While (Not testFeature Is Nothing) And (Not testFeature.Name = lastFeatureName)
        Set featureList(loopCount) = testFeature
        loopCount = loopCount + 1
        Set testFeature = model.FeatureByPositionReverse(loopCount)
    Wend

    vFeatureList = featureList
    Set selMgr = model.SelectionManager
    Set selData = selMgr.CreateSelectData
    longstatus = model.Extension.MultiSelect2((vFeatureList), True, selData)

```

```
    If longstatus > 0 Then
        multiselect_new_features = True
    End If
End Function
```

Specify IGES Levels and Values, Then Import IGES File Example (C#)

This example shows how to:

- specify levels and values for importing IGES data.
- import an IGES file.

```
//-----
// Preconditions: Substitute the path and name
// of your IGES file where noted in the code.
//
// Postconditions:
// 1. Creates a folder named Layer 25.
// 2. Imports the specified IGES file into SOLIDWORKS Design
// and moves the imported IGES features to Layer 25.
// 3. To verify, examine the graphics area and
// FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ImportIGESFilesCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 model = default(ModelDoc2);
            bool boolstatus = false;
            string fileName = null;
            string argString = null;
            ImportIgesData importData = default(ImportIgesData);
            int Err = 0;
            bool orgSetting = false;
            bool allLevels = false;
            object vOnlyLev = null;
            int[] onlyLev = new int[2];
            int oneLev = 0;
            Feature lastFeature = default(Feature);
            Feature newFolder = default(Feature);
            string newFolderName = "";
            string lastFeatureName = null;

            model = (ModelDoc2)swApp.ActiveDoc;
            // Substitute the path and name of your IGES file
            fileName = "C:\\\\beam_with_holes.igs";
            // "r" means open new document
            // "i" means insert into existing document
            if (model == null)
            {
                argString = "r";
                // There is an existing part, so use it
            }
            else
            {
                argString = "i";
            }
            // Fill in the import data
            importData = (ImportIgesData)swApp.GetImportFileData(fileName);
            if ((importData != null))
            {
                // Test the various flags
                importData.IncludeSurfaces = true;
                importData.IncludeCurves = true;
                importData.CurvesAsSketches = true;
                // False = Curves as Curves
                importData.ProcessByLevel = false;
                // Test all levels
                // allLevels = True
                // False = levels specified in vOnlyLev
                // newFolderName = "All levels"
                // or, test multiple levels
                // onlyLev(0) = 0
                // onlyLev(1) = 6
                // vOnlyLev = onlyLev
                // newFolderName = "Layer 0 and 6"
                // Or, test individual levels
                oneLev = 25;
                vOnlyLev = oneLev;
                newFolderName = "Layer " + oneLev.ToString();
                boolstatus = importData.SetLevels(allLevels, (vOnlyLev));
            }

            // Keep the last feature to determine what's been added
            // If this is a new document, that cannot be done, so
            // just hard code the name of the Origin feature,
            // which is currently the last feature in a new part document
            // It is better to always create a new
            // document first, and then call SldWorks::LoadFile4
            // with "i" argString to avoid this potential problem
            if ((model != null))
            {
                lastFeature = (Feature)model.FeatureByPositionReverse(0);
                lastFeatureName = lastFeature.Name;
            }
            else
            {
                lastFeatureName = "Origin";
            }
        }
    }
}
```

```

// Setting this user preference to true means that the IGES
// dialog is displayed
// Setting this user preference to false means that the
// IGES dialog is not displayed, and the import IGES data
// is used if it is passed in, or, if it is not,
// then the default values for the dialog are used
orgSetting = swApp.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swIGESImportShowLevel);
swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swIGESImportShowLevel, false);
model = (ModelDoc2)swApp.LoadFile4(fileName, argString, importData, ref Err);

// swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swIGESImportShowLevel, orgSetting
// If the SldWorks::LoadFile4 failed, do not continue
if (model == null)
{
    Debug.Print("Problem loading file.");
    return;
}
// Retrieve all of the features that were created
// and move them into their own new folder
model.ClearSelection2(true);
// Select features that are then used by FeatureManager::InsertFeatureTreeFolder2
// Either method of selection seems to take the same amount of time
boolstatus = select_new_features_individually(model, lastFeatureName);
// boolstatus = multiselect_new_features(model, lastFeatureName);
if ((boolstatus))
{
    newFolder = model.FeatureManager.InsertFeatureTreeFolder2((int)swFeatureTreeFolderType_e.swFeatureTreeFolder_Containing);
    if ((newFolder != null))
    {
        newFolder.Name = newFolderName;
    }
    model.ClearSelection2(true);
}
}

private bool select_new_features_individually(ModelDoc2 model, string lastFeatureName)
{
    bool functionReturnValue = false;
    Feature testFeature = default(Feature);
    int loopCount = 0;
    bool boolstatus = false;
    functionReturnValue = false;
    loopCount = 0;
    testFeature = (Feature)model.FeatureByPositionReverse(loopCount);
    while (((testFeature != null)) && !(testFeature.Name == lastFeatureName))
    {
        loopCount = loopCount + 1;
        boolstatus = testFeature.Select2(true, 0);
        if (!(boolstatus == false))
        {
            functionReturnValue = true;
        }
        testFeature = (Feature)model.FeatureByPositionReverse(loopCount);
    }
    return functionReturnValue;
}

private bool multiselect_new_features(ModelDoc2 model, string lastFeatureName)
{
    bool functionReturnValue = false;
    Feature testFeature = default(Feature);
    int loopCount = 0;
    Feature[] featureList = null;
    object vFeatureList = null;
    int longstatus = 0;
    SelectData selData = default(SelectData);
    SelectionMgr selMgr = default(SelectionMgr);
    functionReturnValue = false;
    loopCount = 0;
    testFeature = (Feature)model.FeatureByPositionReverse(loopCount);
    while (((testFeature != null)) && !(testFeature.Name == lastFeatureName))
    {
        loopCount = loopCount + 1;
        testFeature = (Feature)model.FeatureByPositionReverse(loopCount);
    }
    Array.Resize(ref featureList, (loopCount + 1));

    loopCount = 0;
    testFeature = (Feature)model.FeatureByPositionReverse(loopCount);
    while (((testFeature != null)) && !(testFeature.Name == lastFeatureName))
    {
        featureList[loopCount] = testFeature;
        loopCount = loopCount + 1;
        testFeature = (Feature)model.FeatureByPositionReverse(loopCount);
    }

    vFeatureList = featureList;
    selMgr = (SelectionMgr)model.SelectionManager;
    selData = (SelectData)selMgr.CreateSelectData();
    longstatus = model.Extension.MultiSelect2((vFeatureList), true, selData);
    if (longstatus > 0)
    {
        functionReturnValue = true;
    }
    return functionReturnValue;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Specify IGES Levels and Values, Then Import IGES File Example (VB.NET)

This example shows how to:

- specify levels and values for importing IGES data.
- import an IGES file.

```

-----
' Preconditions: Substitute the path and name
' of your IGES file where noted in the code.
'
' Postconditions:
' 1. Creates a folder named Layer 25.
' 2. Imports the specified IGES file into SOLIDWORKS Design
'    and moves the imported IGES features to Layer 25.
' 3. To verify, examine the graphics area and
'    FeatureManager design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim model As ModelDoc2
        Dim boolstatus As Boolean
        Dim fileName As String
        Dim argString As String
        Dim importData As ImportIgesData
        Dim Err As Integer
        Dim orgSetting As Boolean
        Dim allLevels As Boolean
        Dim vOnlyLev As Integer
        Dim onlyLev(0 To 1) As Integer
        Dim oneLev As Integer
        Dim lastFeature As Feature
        Dim newFolder As Feature
        Dim newFolderName As String = ""
        Dim lastFeatureName As String

        model = swApp.ActiveDoc
        ' Substitute the path and name of your IGES file
        fileName = "path\IGES_file_name.igs"
        ' "r" means open new document
        ' "i" means insert into existing document
        If model Is Nothing Then
            argString = "r"
        Else
            ' There is an existing part, so use it
            argString = "i"
        End If
        ' Fill in the import data
        importData = swApp.GetImportFileData(fileName)
        If Not importData Is Nothing Then
            ' Test the various flags
            importData.IncludeSurfaces = True
            importData.IncludeCurves = True
            importData.CurvesAsSketches = True ' False = Curves as Curves
            importData.ProcessByLevel = False
            ' Test all levels
            ' allLevels = True
            ' False = levels specified in vOnlyLev
            ' newFolderName = "All levels"
            ' or, test multiple levels
            ' onlyLev(0) = 0
            ' onlyLev(1) = 6
            ' vOnlyLev = onlyLev
            ' newFolderName = "Layer 0 and 6"
            ' Or, test individual levels
            oneLev = 25
            vOnlyLev = oneLev
            newFolderName = "Layer " & Format(oneLev)
            boolstatus = importData.SetLevels(allLevels, (vOnlyLev))
        End If

        ' Keep the last feature to determine what's been added
        ' If this is a new document, that cannot be done, so
        ' just hard code the name of the Origin feature,
        ' which is currently the last feature in a new part document
        ' It is better to always create a new
        ' document first, and then call SldWorks::LoadFile4
        ' with "i" argString to avoid this potential problem
        If Not model Is Nothing Then
            lastFeature = model.FeatureByPositionReverse(0)
            lastFeatureName = lastFeature.Name
        Else
            lastFeatureName = "Origin"
        End If
        ' Setting this user preference to true means that the IGES
        ' dialog is displayed
        ' Setting this user preference to false means that the
        ' IGES dialog is not displayed, and the import IGES data
        ' is used if it is passed in, or, if it is not,
        ' then the default values for the dialog are used
        orgSetting = swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swIGESImportShowLevel)
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swIGESImportShowLevel, False)
        model = swApp.LoadFile4(fileName, argString, importData, Err)

        ' swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swIGESImportShowLevel, orgSetting
    End Sub
End Class

```

```

' If the SldWorks::LoadFile4 failed, do not continue
If model Is Nothing Then
    Debug.Print("Problem loading file. Error message = " & Err.ToString)
    Exit Sub
End If
' Retrieve all of the features that were created
' and move them into their own new folder
model.ClearSelection2(True)
' Select features that are then used by FeatureManager::InsertFeatureTreeFolder2
' Either method of selection seems to take the same amount of time
boolstatus = select_new_features_individually(model, lastFeatureName)
' boolstatus = multiselect_new_features(model, lastFeatureName)
If (boolstatus) Then
    newFolder = model.FeatureManager.InsertFeatureTreeFolder2(swFeatureTreeFolderType_e.swFeatureTreeFolder_Containing)
    If Not newFolder Is Nothing Then
        newFolder.Name = newFolderName
    End If
    model.ClearSelection2(True)
End If
End Sub

Private Function select_new_features_individually(ByVal model As ModelDoc2, ByVal lastFeatureName As String) As Boolean
    Dim testFeature As Feature
    Dim loopCount As Integer
    Dim boolstatus As Boolean
    select_new_features_individually = False
    loopCount = 0
    testFeature = model.FeatureByPositionReverse(loopCount)
    While (Not testFeature Is Nothing) And (Not testFeature.Name = lastFeatureName)
        loopCount = loopCount + 1
        boolstatus = testFeature.Select2(True, 0)
        If Not boolstatus = 0 Then
            select_new_features_individually = True
        End If
        testFeature = model.FeatureByPositionReverse(loopCount)
    End While
End Function

Private Function multiselect_new_features(ByVal model As ModelDoc2, ByVal lastFeatureName As String) As Boolean
    Dim testFeature As Feature
    Dim loopCount As Integer
    Dim featureList() As Feature
    Dim vFeatureList As Object
    Dim longstatus As Integer
    Dim selData As SelectData
    Dim selMgr As SelectionMgr
    multiselect_new_features = False
    loopCount = 0
    testFeature = model.FeatureByPositionReverse(loopCount)
    While (Not testFeature Is Nothing) And (Not testFeature.Name = lastFeatureName)
        loopCount = loopCount + 1
        testFeature = model.FeatureByPositionReverse(loopCount)
    End While
    ReDim featureList(0 To loopCount - 1)
    loopCount = 0
    testFeature = model.FeatureByPositionReverse(loopCount)
    While (Not testFeature Is Nothing) And (Not testFeature.Name = lastFeatureName)
        featureList(loopCount) = testFeature
        loopCount = loopCount + 1
        testFeature = model.FeatureByPositionReverse(loopCount)
    End While

    vFeatureList = featureList
    selMgr = model.SelectionManager
    selData = selMgr.CreateSelectData
    longstatus = model.Extension.MultiSelect2((vFeatureList), True, selData)
    If longstatus > 0 Then
        multiselect_new_features = True
    End If
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Specify Order of Configurations Example (C#)

This example shows how to specify the order in which to list configurations in the ConfigurationManager.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\driveworkspress\mobile gantry.sldasm.
// 2. Click the ConfigurationManager tab and examine the order in which the
//    configurations are listed.
//
// Preconditions:
// 1. Lists the configurations in the specified order.
// 2. Examine the ConfigurationManager tab.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ConfigurationManager swConfigMgr = default(ConfigurationManager);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfigMgr.SortConfigurationTree((int)swConfigTreeSortType_e.swSortType_Literal);
            swModel.EditRebuild3();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Specify Order of Configurations Example (VBA)

This example shows how to specify the order in which to list configurations in the ConfigurationManager.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\driveworksxpess\mobile gantry.sldasm.  
' 2. Click the ConfigurationManager tab and examine the order in which the  
'    configurations are listed.  
'  
' Preconditions:  
' 1. Lists the configurations in the specified order.  
' 2. Examine the ConfigurationManager tab.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swConfigMgr As SldWorks.ConfigurationManager  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swConfigMgr = swModel.ConfigurationManager  
    swConfigMgr.SortConfigurationTree swConfigTreeSortType_e.swSortType_Literal  
    swModel.EditRebuild3  
  
End Sub
```

Specify Order of Configurations Example (VB.NET)

This example shows how to specify the order in which to list configurations in the ConfigurationManager.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\driveworksexpress\mobile gantry.sldasm.
' 2. Click the ConfigurationManager tab and examine the order in which the
'    configurations are listed.
'
' Preconditions:
' 1. Lists the configurations in the specified order.
' 2. Examine the ConfigurationManager tab.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfigMgr As ConfigurationManager

        swModel = swApp.ActiveDoc
        swConfigMgr = swModel.ConfigurationManager
        swConfigMgr.SortConfigurationTree(swConfigTreeSortType_e.swSortType_Literal)
        swModel.EditRebuild3()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Split Closed Sketch Segment Example (C#)

This example shows how to split a closed sketch segment.

```
//-----
// Preconditions:
// 1. Open a new part document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a sketch.
// 2. Creates a closed spline.
// 3. Splits the spline into two segments.
// 4. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SplitClosedSegment_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SketchSegment swSketchSegment;
        object[] skSegmentArray;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swApp.SetUserPreferenceToggle(int swUserPreferenceToggle_e.swSketchInference, false);

            boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.SketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            object pointArray = null;
            double[] points = new double[21];
            points[0] = -0.0243607314462504;
            points[1] = 0.0404155099449417;
            points[2] = 0;
            points[3] = -0.000849151208568891;
            points[4] = 0.0470352752545802;
            points[5] = 0;
            points[6] = 0;
            points[7] = 0.0395024388677502;
            points[8] = 0;
            points[9] = -0.00530037270987752;
            points[10] = 0.0358501545589842;
            points[11] = 0;
            points[12] = -0.00712651486426054;
            points[13] = 0.0283173181721542;
            points[14] = 0;
            points[15] = -0.023219392599761;
            points[16] = 0.0295727909032925;
            points[17] = 0;
            points[18] = -0.0243607314462504;
            points[19] = 0.0404155099449417;
            points[20] = 0;
            pointArray = points;

            swSketchSegment = swModel.SketchManager.CreateSpline((pointArray));
            swModel.ViewZoomtofit2();

            skSegmentArray = (object[])swModel.SketchManager.SplitClosedSegment(-0.0243607314462504, 0.0404155099449417, 0.0,
            -0.023219392599761, 0.0295727909032925, 0.0);

            // Close the sketch and rebuild
            swModel.SketchManager.Insert3DSketch(true);

            Debug.Print("Number of segments in sketch = " + skSegmentArray.Length);
        }
    }

    public SldWorks swApp;
```


Split Closed Sketch Segment Example (VBA)

This example shows how to split a closed sketch segment.

```
'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a sketch.
' 2. Creates a closed spline.
' 3. Splits the spline into two segments.
' 4. Examine the Immediate window.
'-----
```

Option Explicit

```
Dim SwApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchSegment As SldWorks.SketchSegment
Dim skSegmentArray As Variant
Dim boolstatus As Boolean

Sub main()
    Set SwApp = Application.SldWorks
    Set swModel = SwApp.ActiveDoc

    SwApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swSketchInference, False

    boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.SketchManager.InsertSketch True
    swModel.ClearSelection2 True

    Dim pointArray As Variant
    Dim points() As Double
    ReDim points(0 To 20) As Double
    points(0) = -2.43607314462504E-02
    points(1) = 4.04155099449417E-02
    points(2) = 0
    points(3) = -8.49151208568891E-04
    points(4) = 4.70352752545802E-02
    points(5) = 0
    points(6) = 0
    points(7) = 3.95024388677502E-02
    points(8) = 0
    points(9) = -5.30037270987752E-03
    points(10) = 3.58501545589842E-02
    points(11) = 0
    points(12) = -7.12651486426054E-03
    points(13) = 2.83173181721542E-02
    points(14) = 0
    points(15) = -0.023219392599761
    points(16) = 2.95727909032925E-02
    points(17) = 0
    points(18) = -2.43607314462504E-02
    points(19) = 4.04155099449417E-02
    points(20) = 0
    pointArray = points
    Dim skSegment As Object
    Set skSegment = swModel.SketchManager.CreateSpline((pointArray))
    swModel.ViewZoomtofit2

    skSegmentArray = swModel.SketchManager.SplitClosedSegment(-2.43607314462504E-02, 4.04155099449417E-02, 0#,
-0.023219392599761, 2.95727909032925E-02, 0#)

    ' Close the sketch and rebuild
    swModel.SketchManager.Insert3DSketch True

    Debug.Print "Number of segments in sketch = " & UBound(skSegmentArray) + 1
End Sub
```

Split Closed Sketch Segment Example (VB.NET)

This example shows how to split a closed sketch segment.

```
'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a sketch.
' 2. Creates a closed spline.
' 3. Splits the spline into two segments.
' 4. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swSketchSegment As SketchSegment
    Dim skSegmentArray As Object
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchInference, False)

        boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.SketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        Dim pointArray As Object
        Dim points(20) As Double
        points(0) = -0.0243607314462504
        points(1) = 0.0404155099449417
        points(2) = 0
        points(3) = -0.000849151208568891
        points(4) = 0.0470352752545802
        points(5) = 0
        points(6) = 0
        points(7) = 0.0395024388677502
        points(8) = 0
        points(9) = -0.00530037270987752
        points(10) = 0.0358501545589842
        points(11) = 0
        points(12) = -0.00712651486426054
        points(13) = 0.0283173181721542
        points(14) = 0
        points(15) = -0.023219392599761
        points(16) = 0.0295727909032925
        points(17) = 0
        points(18) = -0.0243607314462504
        points(19) = 0.0404155099449417
        points(20) = 0
        pointArray = points

        swSketchSegment = swModel.SketchManager.CreateSpline((pointArray))
        swModel.ViewZoomtofit2()

        skSegmentArray = swModel.SketchManager.SplitClosedSegment(-0.0243607314462504, 0.0404155099449417, 0.0#,
-0.023219392599761, 0.0295727909032925, 0.0#)

        ' Close the sketch and rebuild
        swModel.SketchManager.Insert3DSketch(True)
        Debug.Print("Number of segments in sketch = " & UBound(skSegmentArray) + 1)

    End Sub

    Public swApp As SldWorks

End Class
```

Split FeatureManager Design Tree and Position Splitter Example (C#)

This example shows how:

- to split a FeatureManager design tree.
- add a tab to one of the FeatureManager design trees.
- change the location of the split panel bar (splitter).

```
//-----
// Preconditions:
// 1. In the IDE, reference your ActiveX control file
//    (click Project > Add Reference > Browse and browse
//    to the folder where the ActiveX control resides and click
//    the ActiveX control file > OK).
// 2. Verify that the specified part document and bitmap exist.
// 3. Replace activex_control_component_declaration and
//    activex_control_CLSID_or_ProgID with your ActiveX control's
//    information.
//
// Postconditions:
// 1. Opens the part document and splits the FeatureManager
//    design tree; the splitter is below the FeatureManager
//    design tree to which the tab was added. Drag the splitter
//    to verify.
// 2. Close the part document.
// 3. Set test to 1.
// 4. Rerun the macro.
// 5. Opens the part document and splits the FeatureManager
//    design tree; the splitter is above the FeatureManager
//    design tree to which the tab was added. Drag the
//    splitter to verify.
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace FeatureManagerSplitterPositionModelDoc2CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            const string iconSmall = "C:\\Program Files\\SOLIDWORKS Corp\\SOLIDWORKS\\data\\user macro icons\\button.bmp";

            ModelDoc2 swModel = default(ModelDoc2);
            ModelViewManager swModViewMgr = default(ModelViewManager);
            FeatMgrView swFeatMgrTabTop = default(FeatMgrView);
            FeatMgrView swFeatMgrTabBtm = default(FeatMgrView);
            activex_control_component_declaration ctrlTop = default(activex_control_component_declaration);
            activex_control_component_declaration ctrlBtm = default(activex_control_component_declaration);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            int activePane = 0;
            int test = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\fillets\\knob.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModViewMgr = (ModelViewManager)swModel.ModelViewManager;

            if (test == 0)
            {
                // FeatureManager design tree is split, and the splitter is
                // below the FeatureManager design tree to which the
                // tab was added
                swFeatMgrTabTop = (FeatMgrView)swModViewMgr.CreateFeatureMgrControl2(iconSmall,
                "activex_control_CLSID_or_ProgID", "", "Top tab ToolTip", (int)swFeatMgrPane_e.swFeatMgrPaneTop);
                ctrlTop = (activex_control_component_declaration)swFeatMgrTabTop.GetControl();
                swModel.FeatureManagerSplitterPosition = 0.0;
                activePane = swFeatMgrTabTop.ActivateView();
            }
            else
            {
                // FeatureManager design tree is split, and the splitter is
                // above the FeatureManager design tree to which the
                // tab was added
                swFeatMgrTabBtm = (FeatMgrView)swModViewMgr.CreateFeatureMgrControl2(iconSmall,
                "activex_control_CLSID_or_ProgID", "", "Bottom tab ToolTip", (int)swFeatMgrPane_e.swFeatMgrPaneBottom);
                ctrlBtm = (activex_control_component_declaration)swFeatMgrTabBtm.GetControl();
                swModel.FeatureManagerSplitterPosition = 1.0;
                activePane = swFeatMgrTabBtm.ActivateView();
            }
        }
    }
}
```

```
}  
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}  
}
```

Split FeatureManager Design Tree and Position Splitter Example (VBA)

This example shows how:

- to split a FeatureManager design tree.
- add a tab to one of the FeatureManager design trees.
- change the location of the split panel bar (splitter).

```

'-----
' Preconditions:
' 1. In the IDE:
'   a. Reference your ActiveX control file (click Tools > References > Browse > browse to the folder where the
'      ActiveX control file resides and click the ActiveX control file > Open > OK).
'   a. Insert the ActiveX component (click Insert > Components > ActiveX component's check box > OK).
' 2. Verify that the specified part document and bitmap exist.
' 3. Replace activex_control_component_declaration and activex_control_CLSID_or_ProgID with your ActiveX control's
'    information.
'
' Postconditions:
' 1. Opens the part document and splits the FeatureManager design tree; the splitter is below the FeatureManager
'    design tree to which the tab was added. Drag the splitter to verify.
' 2. Close the part document.
' 3. Set test to 1 in the macro.
' 4. Rerun the macro.
' 5. Opens the part document and splits the FeatureManager design tree; the splitter is above the FeatureManager
'    design tree to which the tab was added. Drag the splitter to verify.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----

```

Option Explicit

```

Sub main()
    Const iconSmall As String = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\user macro icons\button.bmp"

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModViewMgr As SldWorks.ModelViewManager
    Dim swFeatMgrTabTop As SldWorks.FeatMgrView
    Dim swFeatMgrTabBtm As SldWorks.FeatMgrView
    Dim ctrlTop As activex_control_component_declaration
    Dim ctrlBtm As activex_control_component_declaration
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long
    Dim activePane As Long
    Dim test As Long

    Set swApp = CreateObject("SldWorks.Application")
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swModViewMgr = swModel.ModelViewManager

    test = 0

    If test = 0 Then
        ' FeatureManager design tree is split, and the splitter is
        ' below the FeatureManager design tree to which the
        ' tab was added
        Set swFeatMgrTabTop = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Top tab ToolTip",
swFeatMgrPaneTop)
        Set ctrlTop = swFeatMgrTabTop.GetControl
        swModel.FeatureManagerSplitterPosition = 0#
        activePane = swFeatMgrTabTop.ActivateView
    Else
        ' FeatureManager design tree is split, and the splitter is
        ' above the FeatureManager design tree to which the
        ' tab was added
        Set swFeatMgrTabBtm = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Bottom tab ToolTip",
swFeatMgrPaneBottom)
        Set ctrlBtm = swFeatMgrTabBtm.GetControl
        swModel.FeatureManagerSplitterPosition = 1#
        activePane = swFeatMgrTabBtm.ActivateView
    End If
End Sub

```

Split FeatureManager Design Tree and Position Splitter Example (VB.NET)

This example shows how:

- to split a FeatureManager design tree.
- add a tab to one of the FeatureManager design trees.
- change the location of the split panel bar (splitter).

```

'-----
' Preconditions:
' 1. In the IDE, reference your ActiveX control file
'    (click Project > Add Reference > Browse and browse
'    to the folder where the ActiveX control resides and click
'    the ActiveX control file > OK).
' 2. Verify that the specified part document and bitmap exist.
' 3. Replace activex_control_component_declaration and
'    activex_control_CLSID_or_ProgID with your ActiveX control's
'    information.
'
' Postconditions:
' 1. Opens the part document and splits the FeatureManager design
'    tree; the splitter is below the FeatureManager design tree
'    to which the tab was added. Drag the splitter to verify.
' 2. Close the part document.
' 3. Set test to 1 in the macro.
' 4. Rerun the macro.
' 5. Opens the part document and splits the FeatureManager design
'    tree; the splitter is above the FeatureManager design tree
'    to which the tab was added. Drag the splitter to verify.
'
' NOTE: Because the part document is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst

Partial Class SolidWorksMacro

    Public Sub Main()

        Const iconSmall As String = "C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\data\user macro icons\button.bmp"

        Dim swModel As ModelDoc2
        Dim swModViewMgr As ModelViewManager
        Dim swFeatMgrTabTop As FeatMgrView
        Dim swFeatMgrTabBtm As FeatMgrView
        Dim ctrlTop As activex_control_component_declaration
        Dim ctrlBtm As activex_control_component_declaration
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim activePane As Integer
        Dim test As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\fillets\knob.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)
        swModViewMgr = swModel.ModelViewManager

        test = 0

        If test = 0 Then
            ' FeatureManager design tree is split, and the splitter is
            ' below the FeatureManager design tree to which the
            ' tab was added
            swFeatMgrTabTop = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Top
tab Tooltip", swFeatMgrPane_e.swFeatMgrPaneTop)
            ctrlTop = swFeatMgrTabTop.GetControl
            swModel.FeatureManagerSplitterPosition = 0.0#
            activePane = swFeatMgrTabTop.ActivateView
        Else
            ' FeatureManager design tree is split, and the splitter is
            ' above the FeatureManager design tree to which the
            ' tab was added
            swFeatMgrTabBtm = swModViewMgr.CreateFeatureMgrControl2(iconSmall, "activex_control_CLSID_or_ProgID", "", "Bottom
tab Tooltip", swFeatMgrPane_e.swFeatMgrPaneBottom)
            ctrlBtm = swFeatMgrTabBtm.GetControl
            swModel.FeatureManagerSplitterPosition = 1.0#
            activePane = swFeatMgrTabBtm.ActivateView
        End If

    End Sub

''' <summary>

```

```
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Split Open Sketch Segment Example (C#)

This example shows how to split an open sketch segment.

```
//-----
// Preconditions:
// 1. Open a new part document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a sketch.
// 2. Creates a line segment.
// 3. Splits the line segment into two segments.
// 4. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SplitOpenSegment_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SketchSegment swSketchSegment;
        object[] skSegmentArray;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swApp.SetUserPreferenceToggle(int swUserPreferenceToggle_e.swSketchInference, false);

            boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.SketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            // Create a line
            swSketchSegment = swModel.SketchManager.CreateLine(-0.055964, 0.033212, 0.0, 0.102938, -0.014129, 0.0);
            swModel.ViewZoomtofit2();

            skSegmentArray = (object[])swModel.SketchManager.SplitOpenSegment(0.02, 0.01, 0.0);

            // Close the sketch and rebuild
            swModel.SketchManager.Insert3DSketch(true);
            Debug.Print("Number of segments in sketch = " + skSegmentArray.Length);
        }

        public SldWorks swApp;
    }
}
```

Split Open Sketch Segment Example (VBA)

This example shows how to split an open sketch segment.

```
'-----  
' Preconditions:  
' 1. Open a new part document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens a sketch.  
' 2. Creates a line segment.  
' 3. Splits the line segment into two segments.  
' 4. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim SwApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim skSegmentArray As Variant  
Dim boolstatus As Boolean  
  
Sub main()  
    Set SwApp = Application.SldWorks  
    Set swModel = SwApp.ActiveDoc  
  
    SwApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swSketchInference, False  
    boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
    swModel.SketchManager.InsertSketch True  
    swModel.ClearSelection2 True  
  
    'Create a line  
    Set swSketchSegment = swModel.SketchManager.CreateLine(-0.055964, 0.033212, 0#, 0.102938, -0.014129, 0#)  
    swModel.ViewZoomtofit2  
  
    skSegmentArray = swModel.SketchManager.SplitOpenSegment(0.02, 0.01, 0#)  
    ' Close the 3D sketch and rebuild  
    swModel.SketchManager.Insert3DSketch True  
    Debug.Print "Number of sketch segments = " & UBound(skSegmentArray) + 1  
  
End Sub
```

Split Open Sketch Segment Example (VB.NET)

This example shows how to split an open sketch segment.

```
'-----
' Preconditions:
' 1. Open a new part document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a sketch.
' 2. Creates a line segment.
' 3. Splits the line segment into two segments.
' 4. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swAssembly As AssemblyDoc
    Dim swSketchMgr As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim skSegmentArray As Object
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchInference, False)

        boolstatus = swModel.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.SketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        ' Create a line
        swSketchSegment = swModel.SketchManager.CreateLine(-0.055964, 0.033212, 0.0#, 0.102938, -0.014129, 0.0#)
        swModel.ViewZoomtofit2()

        skSegmentArray = swModel.SketchManager.SplitOpenSegment(0.02, 0.01, 0.0#)

        ' Close the sketch and rebuild
        swModel.SketchManager.Insert3DSketch(True)
        Debug.Print("Number of segments in sketch = " & UBound(skSegmentArray) + 1)

    End Sub

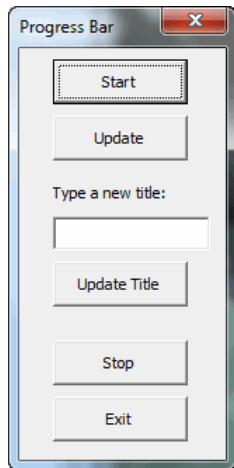
    Public swApp As SldWorks

End Class
```

Start, Update, and Stop Progress Indicator Example (C#)

This example shows how to create, start, update, and stop a progress indicator on the system task bar.

```
//-----
// Preconditions:
// 1. Create a C# macro in SOLIDWORKS Design called ManipulatePB_CSharp.
// 2. Right-click the macro project name in the Project Explorer and click
//    Add > Windows Form.
// 3. Click Add.
// 4. Modify Form1.cs [Design] to look like this:
```



```
// 5. Copy Forms - Form1 into Form1.cs.
// 6. Modify control_name in each control_name_Click subroutine to
//    match your form.
// 7. Copy SolidWorksMacro into SolidWorksMacro.cs.
// 8. Open the Immediate window.
//
// Postconditions:
// 1. Shows a dialog box.
// 2. Click Start.
//    * Displays a progress indicator on the SOLIDWORKS Design icon on the
//      system taskbar.
//    * Increments the progress indicator to 100% completion.
// 3. Click Update repeatedly to increment the progress indicator in steps of
//    10.
// 4. Inspect the Immediate window to see the return code after each update.
// 5. Type a new title and click Update Title to change the progress indicator
//    title.
// 6. Click Stop to remove the progress indicator from SOLIDWORKS Design.
// 7. Click Exit to close the dialog box.
//-----
```

Forms - Form1

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using Microsoft.VisualBasic;

namespace ManipulateUserPB_CSharp.csproj
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            UserProgressBar pb;
            int Position;
            int lRet;
            bool retVal;
```

```

bool boolstatus;
public SldWorks swApp;

public void cmdExit_Click(System.Object sender, System.EventArgs e)
{
    this.Close();
}

public void cmdPBTitle_Click(System.Object sender, System.EventArgs e)
{
    retVal = pb.UpdateTitle(textBox1.Text);
}

public void cmdStartPB_Click(System.Object sender, System.EventArgs e)
{
    boolstatus = pb.Start(0, 160, "Status");
    while (!(Position == 160)) {
        Position = Position + 10;
        lRet = pb.UpdateProgress(Position);
    }
    Position = 0;
}

public void cmdStopPB_Click(System.Object sender, System.EventArgs e)
{
    pb.End();
}

public void Form1_Initialize(System.Object sender, System.EventArgs e)
{
    swApp = (SldWorks)System.Runtime.InteropServices.Marshal.GetActiveObject("SldWorks.Application");
    retVal = swApp.GetUserProgressBar(out pb);
}

public void cmdUpdatePB_Click(System.Object sender, System.EventArgs e)
{
    Position = Position + 10;
    if ((Position == 160))
        Position = 0;
    lRet = pb.UpdateProgress(Position);
    if (lRet != 2) {
        Debug.Print(" Result " + lRet);
    } else {
        MessageBox.Show(" User pressed Esc to cancel ", " API");
        pb.End();
    }
}

private void TextBox1_TextChanged(System.Object sender, System.EventArgs e)
{
}
}
}

```

[Back to top](#)

SolidWorksMacro

```

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace ManipulateUserPB_CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        System.Windows.Forms.Form aForm;
        public void Main()
        {
            aForm = new Form1();
            aForm.ShowDialog();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}

```

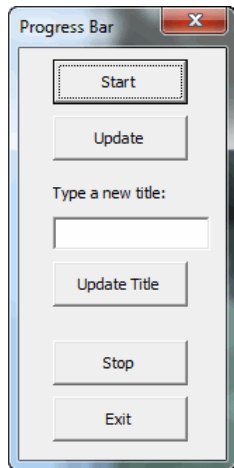
[Back to top](#)

Start, Update, and Stop Progress Indicator Example (VB.NET)

This example shows how to create, start, update, and stop a progress indicator on the system task bar.

Preconditions:

1. Create a VB.NET macro in SOLIDWORKS Design .
2. Right-click the macro project name in the Project Explorer and click **Add > Windows Form**.
3. Click **Add**.
4. Modify **Form1.vb [Design]** to look like this:



5. Copy [Forms - Form1](#) into **Form1.vb**.
6. Modify *control_name* in each *control_name_Click* subroutine to match your form.
7. Copy [SolidWorksMacro](#) into **SolidWorksMacro.vb**.
8. Open the Immediate window.

Postconditions:

1. Shows a dialog box.
2. Click **Start**.
 - * Displays a progress indicator on the SOLIDWORKS Design icon on the system taskbar.
 - * Increments the progress indicator to 100% completion.
3. Click **Update** repeatedly to increment the progress indicator in steps of 10.
4. Inspect the Immediate window to see the return code after each update.
5. Type a new title and click **Update Title** to change the progress indicator title.
6. Click **Stop** to remove the progress indicator from SOLIDWORKS Design.
7. Click **Exit** to close the dialog box.

Forms - Form1

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Public Class Form1
    Dim pb As UserProgressBar
    Dim Position As Integer
    Dim lRet As Integer
    Dim retVal As Boolean
    Dim boolstatus As Boolean
    Public swApp As SldWorks

    Public Sub cmdExit_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles cmdExit.Click
        Me.Close()
    End Sub

    Public Sub cmdPBTitle_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles cmdPBTitle.Click
        retVal = pb.UpdateTitle(TextBox1.Text)
    End Sub

    Public Sub cmdStartPB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles cmdStartPB.Click
        boolstatus = pb.Start(0, 160, "Status")
        Do While Not (Position = 160)
```

```

        Position = Position + 10
        lRet = pb.UpdateProgress(Position)
    Loop
    Position = 0
End Sub

```

```

Public Sub cmdStopPB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles cmdStopPB.Click
    pb.End()
End Sub

```

```

Public Sub Form1_Initialize(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles Me.Load
    swApp = CType(System.Runtime.InteropServices.Marshal.GetActiveObject("SldWorks.Application"), SldWorks)
    retVal = swApp.GetUserProgressBar(pb)
End Sub

```

```

Public Sub cmdUpdatePB_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles cmdUpdatePB.Click
    Position = Position + 10
    If (Position = 160) Then Position = 0
    lRet = pb.UpdateProgress(Position)
    If lRet <> 2 Then
        Debug.Print(" Result " & lRet)
    Else
        MsgBox(" User pressed Esc to cancel ", vbInformation, " API")
        pb.End()
    End If
End Sub

```

```

Private Sub TextBox1_TextChanged(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles TextBox1.TextChanged
End Sub

```

```
End Class
```

[Back to top](#)

SolidWorksMacro

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```

Partial Class SolidWorksMacro
    Dim aForm As Windows.Forms.Form
    Public Sub main()
        aForm = New Form1
        aForm.ShowDialog()
    End Sub

```

```

    Public swApp As SldWorks

```

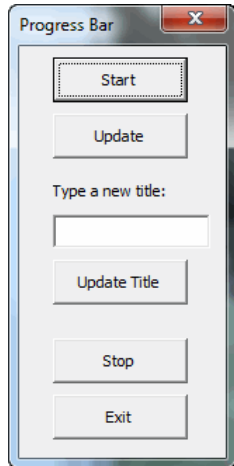
```
End Class
```

[Back to top](#)

Start, Update, and Stop Progress Indicator Example (VBA)

This example shows how to create, start, update, and stop a progress indicator on the status bar.

- Preconditions:
1. Create a VBA macro in SOLIDWORKS Design.
 2. Right-click the macro project name in the Project Explorer and click **Insert > UserForm**.
 3. Modify **UserForm1 (UserForm)** to look like this:



4. Copy [Forms - UserForm1](#) into **UserForm1 (Code)**.
 5. Modify *control_name* in each *control_name_Click* subroutine to match your form.
 6. Right-click the macro project name and click **Insert > Module**.
 7. Copy [Modules](#) into the new module.
 8. Open the Immediate window.
- Postconditions:
1. Shows a dialog box.
 2. Click **Start**.
 - * Displays a progress indicator on the SOLIDWORKS Design status bar.
 - * Increments the progress indicator to 100% completion.
 3. Click **Update** repeatedly to increment the progress indicator in steps of 10.
 4. Inspect the Immediate window to see the return code after each update.
 5. Type a new title and click **Update Title** to change the progress indicator title.
 6. Click **Stop** to remove the progress indicator from the SOLIDWORKS Design status bar.
 7. Click **Exit** to close the dialog box.

Forms - UserForm1

```

Dim swApp As SldWorks.SldWorks
Dim pb As SldWorks.UserProgressBar
Dim Position As Long
Dim lRet As Long
Dim retVal As Boolean
Dim boolstatus As Boolean

Public Sub cmdExit_Click()
    Unload Me
End Sub

Public Sub cmdPBTitle_Click()
    retVal = pb.UpdateTitle(TextBox1.Text)
End Sub

Public Sub cmdStartPB_Click()
    boolstatus = pb.Start(0, 160, "Status")
    Do While Not (Position = 160)
        Position = Position + 10
        lRet = pb.UpdateProgress(Position)
    Loop

```

```

        Position = 0
End Sub

Public Sub cmdStopPB_Click()
    pb.End
End Sub

Public Sub UserForm_Initialize()
    Set swApp = Application.SldWorks
    retVal = swApp.GetUserProgressBar(pb)
End Sub

Public Sub cmdUpdatePB_Click()
    Position = Position + 10
    If (Position = 160) Then Position = 0
    lRet = pb.UpdateProgress(Position)
    If lRet <> 2 Then
        Debug.Print " Result " & lRet
    Else
        MsgBox " User pressed Esc to cancel ", vbInformation, " API"
        pb.End
        End
    End If
End Sub

```

[Back to top](#)

Modules

```

Dim swApp As SldWorks.SldWorks
Sub main()
    UserForm1.Show vbModeless
End Sub

```

[Back to top](#)

Suppress Component Feature Example (C#)

This example shows how to suppress the selected component feature.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Select a component feature in the FeatureManager
// design tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Suppresses the selected component feature.
// 2. Prints the names of the assembly and the suppressed
// component feature to the Immediate window.
// 3. Examine the Immediate window.
//
// NOTE: Editing a component requires that geometry on
// the component to be selected. However, not
// all features are associated with geometry.
// Therefore, it is necessary to select the component
// before attempting to edit the component.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SelectFeatureCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            ModelDoc2 swEditModel = default(ModelDoc2);
            PartDoc swEditPart = default(PartDoc);
            AssemblyDoc swEditAssy = default(AssemblyDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            Component2 swComp = default(Component2);
            string featName = null;
            int status = 0;
            int info = 0;
            bool retVal = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            Debug.Assert((swFeat != null));
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent2(1);
            Debug.Assert((swComp != null));

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swFeat.Name + " <" + swFeat.GetTypeName() + ">");
            Debug.Print("");
            featName = swFeat.Name;
            retVal = swComp.Select2(false, 0);
            Debug.Assert(retVal);
            status = swAssy.EditPart2(true, false, ref info);
            Debug.Assert((int)swEditPartCommandStatus_e.swEditPartSuccessful == status);
            swEditModel = (ModelDoc2)swAssy.GetEditTarget();
            switch (swEditModel.GetType())
            {
                case (int)swDocumentTypes_e.swDocPART:
                    swEditPart = (PartDoc)swEditModel;
                    swFeat = (Feature)swEditPart.FeatureByName(featName);
                    Debug.Assert((swFeat != null));
                    retVal = swFeat.Select2(false, 0);
                    Debug.Assert(retVal);
                    break;
                case (int)swDocumentTypes_e.swDocASSEMBLY:
                    swEditAssy = (AssemblyDoc)swEditModel;
                    swFeat = (Feature)swEditAssy.FeatureByName(featName);
                    Debug.Assert((swFeat != null));
                    retVal = swFeat.Select2(false, 0);
                    Debug.Assert(retVal);
                    break;
                default:
                    Debug.Assert(false);
                    break;
            }
            // Suppress the selected feature
            retVal = swEditModel.EditSuppress2();
            Debug.Assert(retVal);
            swAssy.EditAssembly();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Suppress Component Feature Example (VBA)

This example shows how to suppress the selected component feature.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Select a component feature in the FeatureManager
'    design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Suppressed the selected component feature.
' 2. Prints the names of the assembly and the suppressed
'    component feature to the Immediate window.
' 3. Examine the Immediate window.
'
' NOTE: Editing a component requires that geometry on
' the component be selected. However, not
' all features are associated with geometry.
' Therefore, it is necessary to select the component
' before attempting to edit the component.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swEditModel As SldWorks.ModelDoc2
Dim swEditPart As SldWorks.PartDoc
Dim swEditAssy As SldWorks.AssemblyDoc
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swComp As SldWorks.Component2
Dim featName As String
Dim status As Long
Dim info As Long
Dim retVal As Boolean

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swAssy = swModel
Set swSelMgr = swModel.SelectionManager

Set swFeat = swSelMgr.GetSelectedObject6(1, -1): Debug.Assert Not swFeat Is Nothing
Set swComp = swSelMgr.GetSelectedObjectsComponent2(1): Debug.Assert Not swComp Is Nothing

Debug.Print "File = " & swModel.GetPathName
Debug.Print " " & swFeat.Name & " <" & swFeat.GetTypeName & ">"
Debug.Print ""

featName = swFeat.Name

retVal = swComp.Select2(False, 0): Debug.Assert retVal
status = swAssy.EditPart2(True, False, info): Debug.Assert swEditPartSuccessful = status

Set swEditModel = swAssy.GetEditTarget

Select Case swEditModel.GetType
    Case swDocPART
        Set swEditPart = swEditModel
        Set swFeat = swEditPart.FeatureByName(featName): Debug.Assert Not swFeat Is Nothing
        retVal = swFeat.Select2(False, 0): Debug.Assert retVal

    Case swDocASSEMBLY
        Set swEditAssy = swEditModel
        Set swFeat = swEditAssy.FeatureByName(featName): Debug.Assert Not swFeat Is Nothing
        retVal = swFeat.Select2(False, 0): Debug.Assert retVal

    Case Else
        Debug.Assert False

End Select

' Suppress the selected feature
retVal = swEditModel.EditSuppress2: Debug.Assert retVal
swAssy.EditAssembly
```

End Sub

Suppress Component Feature Example (VB.NET)

This example shows how to suppress the selected component feature.

```

'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Select a component feature in the FeatureManager
'    design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Suppresses the selected component feature.
' 2. Prints the names of the assembly and the suppressed
'    component feature to the Immediate window.
' 3. Examine the Immediate window.
'
' NOTE: Editing a component requires that geometry on
' the component be selected. However, not
' all features are associated with geometry.
' Therefore, it is necessary to select the component
' before attempting to edit the component.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim swEditModel As ModelDoc2
        Dim swEditPart As PartDoc
        Dim swEditAssy As AssemblyDoc
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swComp As Component2
        Dim featName As String
        Dim status As Integer
        Dim info As Integer
        Dim retVal As Boolean

        swModel = swApp.ActiveDoc
        swAssy = swModel
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1) : Debug.Assert(Not swFeat Is Nothing)
        swComp = swSelMgr.GetSelectedObjectsComponent2(1) : Debug.Assert(Not swComp Is Nothing)

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swFeat.Name & " <" & swFeat.GetTypeName & ">")
        Debug.Print("")

        featName = swFeat.Name
        retVal = swComp.Select2(False, 0) : Debug.Assert(retVal)
        status = swAssy.EditPart2(True, False, info) : Debug.Assert(swEditPartCommandStatus_e.swEditPartSuccessful = status)
        swEditModel = swAssy.GetEditTarget

        Select Case swEditModel.GetType
            Case swDocumentTypes_e.swDocPART
                swEditPart = swEditModel
                swFeat = swEditPart.FeatureByName(featName) : Debug.Assert(Not swFeat Is Nothing)
                retVal = swFeat.Select2(False, 0) : Debug.Assert(retVal)
            Case swDocumentTypes_e.swDocASSEMBLY
                swEditAssy = swEditModel
                swFeat = swEditAssy.FeatureByName(featName) : Debug.Assert(Not swFeat Is Nothing)
                retVal = swFeat.Select2(False, 0) : Debug.Assert(retVal)
            Case Else
                Debug.Assert(False)
        End Select

        ' Suppress the selected feature;
        retVal = swEditModel.EditSuppress2 : Debug.Assert(retVal)
        swAssy.EditAssembly()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Suppress Feature Example (VBA)

This example shows how to suppress the selected feature.

```
'-----
'
' Preconditions:
'   (1) Part or assembly is open.
'   (2) At least one entity is selected.
'
' Postconditions: All features associated with the selected
'   entities are suppressed.
'
' NOTE: Features are only suppressed if
'   the owning feature exists for that entity.
'   This typically occurs for edges.
'-----

Option Explicit
Public Enum swSelectType_e
    swSelNOTHING = 0
    swSelEDGES = 1      ' "EDGE"
    swSelFACES = 2      ' "FACE"
    swSelVERTICES = 3    ' "VERTEX"
    swSelDATUMPLANES = 4 ' "PLANE"
    swSelDATUMAXES = 5   ' "AXIS"
    swSelDATUMPOINTS = 6 ' "DATUMPOINT"
    swSelOLEITEMS = 7    ' "OLEITEM"
    swSelATTRIBUTES = 8  ' "ATTRIBUTE"
    swSelSKETCHES = 9    ' "SKETCH"
    swSelSKETCHSEGS = 10 ' "SKETCHSEGMENT"
    swSelSKETCHPOINTS = 11 ' "SKETCHPOINT"
    swSelDRAWINGVIEWS = 12 ' "DRAWINGVIEW"
    swSelGTOLS = 13      ' "GTOL"
    swSelDIMENSIONS = 14 ' "DIMENSION"
    swSelNOTES = 15      ' "NOTE"
    swSelSECTIONLINES = 16 ' "SECTIONLINE"
    swSelDETAILCIRCLES = 17 ' "DETAILCIRCLE"
    swSelSECTIONTEXT = 18 ' "SECTIONTEXT"
    swSelSHEETS = 19     ' "SHEET"
    swSelCOMPONENTS = 20 ' "COMPONENT"
    swSelMATES = 21      ' "MATE"
    swSelBODYFEATURES = 22 ' "BODYFEATURE"
    swSelREFCURVES = 23  ' "REFCURVE"
    swSelEXTSKETCHSEGS = 24 ' "EXTSKETCHSEGMENT"
    swSelEXTSKETCHPOINTS = 25 ' "EXTSKETCHPOINT"
    swSelHELIX = 26      ' "HELIX" (is this wrong?)
    swSelREFERENCECURVES = 26 ' "REFERENCECURVES"
    swSelREFSURFACES = 27 ' "REFSURFACE"
    swSelCENTERMARKS = 28 ' "CENTERMARKS"
    swSelINCONTEXTFEAT = 29 ' "INCONTEXTFEAT"
    swSelMATEGROUP = 30  ' "MATEGROUP"
    swSelBREAKLINES = 31 ' "BREAKLINE"
    swSelINCONTEXTFEATS = 32 ' "INCONTEXTFEATS"
    swSelMATEGROUPS = 33 ' "MATEGROUPS"
    swSelSKETCHTEXT = 34 ' "SKETCHTEXT"
    swSelSFSYMBOLS = 35  ' "SFSYMBOL"
    swSelDATUMTAGS = 36  ' "DATUMTAG"
    swSelCOMPPATTERN = 37 ' "COMPPATTERN"
    swSelWELDS = 38      ' "WELD"
    swSelCTHREADS = 39   ' "CTHREAD"
    swSelDTMTARGS = 40   ' "DTMTARG"
    swSelPOINTREFS = 41  ' "POINTREF"
    swSelDCABINETS = 42  ' "DCABINET"
```

```

swSelEXPLVIEWS = 43      ' "EXPLODEDVIEWS"
swSelEXPLSTEPS = 44      ' "EXPLODESTEPS"
swSelEXPLLINES = 45      ' "EXPLODELINES"
swSelSILHOUETTES = 46    ' "SILHOUETTE"
swSelCONFIGURATIONS = 47 ' "CONFIGURATIONS"
swSelOBJHANDLES = 48
swSelARROWS = 49         ' "VIEWARROW"
swSelZONES = 50          ' "ZONES"
swSelREFEDGES = 51       ' "REFERENCE-EDGE"
swSelREFFACES = 52
swSelREFSILHOUETTE = 53
swSelBOMS = 54           ' "BOM"
swSelEQNFOLDER = 55      ' "EQNFOLDER"
swSelSKETCHHATCH = 56    ' "SKETCHHATCH"
swSelIMPORTFOLDER = 57   ' "IMPORTFOLDER"
swSelVIEWERHYPERLINK = 58 ' "HYPERLINK"
swSelMIDPOINTS = 59
swSelCUSTOMSYMBOLS = 60  ' "CUSTOMSYMBOL"
swSelCOORDSYS = 61       ' "COORDSYS"
swSelDATUMLINES = 62     ' "REFLINE"
swSelROUTECURVES = 63
swSelBOMTEMPS = 64       ' "BOMTEMP"
swSelROUTEPOINTS = 65    ' "ROUTEPOINT"
swSelCONNECTIONPOINTS = 66 ' "CONNECTIONPOINT"
swSelROUTESWEEPS = 67
swSelPOSGROUP = 68       ' "POSGROUP"
swSelBROWSERITEM = 69    ' "BROWSERITEM"
swSelFABRICATEDROUTE = 70 ' "ROUTEFABRICATED"
swSelSKETCHPOINTFEAT = 71 ' "SKETCHPOINTFEAT"
swSelEMPTYSPACE = 72     ' (is this wrong?)
swSelCOMPSDONT OVERRIDE = 72
swSelLIGHTS = 73         ' "LIGHTS"
swSelWIREBODIES = 74
swSelSURFACEBODIES = 75  ' "SURFACEBODY"
swSelSOLIDBODIES = 76    ' "SOLIDBODY"
swSelFRAMEPOINT = 77     ' "FRAMEPOINT"
swSelSURFBODIESFIRST = 78
swSelMANIPULATORS = 79   ' "MANIPULATOR"
swSelPICTUREBODIES = 80  ' "PICTURE BODY"
swSelSOLIDBODIESFIRST = 81
swSelDOWELSYMS = 86      ' "DOWELSYM"
swSelEXTSKETCHTEXT = 88  ' "EXTSKETCHTEXT"
swSelBLOCKINST = 93      ' "BLOCKINST"
swSelFTRFOLDER = 94      ' "FTRFOLDER"
swSelSKETCHREGION = 95   ' "SKETCHREGION"
swSelSKETCHCONTOUR = 96  ' "SKETCHCONTOUR"
swSelBOMFEATURES = 97    ' "BOMFEATURE"
swSelANNOTATIONTABLES = 98 ' "ANNOTATIONTABLES"
swSelBLOCKDEF = 99       ' "BLOCKDEF"
swSelCENTERMARKSYMS = 100 ' "CENTERMARKSYMS"
swSelCENTERLINES = 103   ' "CENTERLINE"
swSelHOLETABLEFEATS = 104 ' "HOLETABLE"
swSelHOLETABLEAXES = 105 ' "HOLETABLEAXIS"
swSelWELDMENT = 106      ' "WELDMENT"
swSelSUBWELDFOLDER = 107 ' "SUBWELDMENT"
swSelEXCLUDEMANIPULATORS = 111
swSelREVISIONTABLE = 113 ' "REVISIONTABLE"
swSelBODYFOLDER = 118    ' "BDYFOLDER"
swSelREVISIONTABLEFEAT = 119
swSelEVERYTHING = -3
swSelLOCATIONS = -2
swSelUNSUPPORTED = -1
' swSelEVERYTHING = 4294967293
' swSelLOCATIONS = 4294967294
' swSelUNSUPPORTED = 4294967295

```

```
End Enum
```

```
Sub main()
```

```
Dim swApp As SldWorks.SldWorks
```

```
Dim swModel          As SldWorks.ModelDoc2
Dim swSelMgr          As SldWorks.SelectionMgr
Dim swFeat            As SldWorks.Feature
Dim i                 As Long
Dim bRet              As Boolean
```

```
On Error Resume Next
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
```

```
Debug.Print "File = " & swModel.GetPathName
```

```
' Information about selected entities
```

```
For i = 1 To swSelMgr.GetSelectedObjectCount
    Set swFeat = swSelMgr.GetSelectedObject5(i)
```

```
    Debug.Print " SelType(" & i & ") = " & swSelMgr.GetSelectedObjectType(i)
```

```
    If Not swFeat Is Nothing Then
```

```
        Debug.Print " " & swFeat.Name & " <" & swFeat.GetTypeName & ">"
```

```
    End If
```

```
Next i
```

```
Debug.Print ""
```

```
' Suppress what has been selected
```

```
bRet = swModel.EditSuppress2: Debug.Assert bRet
```

```
End Sub
```

```
'-----
```

Suppress Rebuild Error Dialogs Example (VBA)

This example shows how to suppress dialogs when rebuild errors occur.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Examine the Immediate window.  
' 2. Change False to True.  
' 3. Rerun the macro.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Set swApp = Application.SldWorks  
    Debug.Print "ShowErrorsEveryRebuild = " & swApp.GetUserPreferenceToggle(swShowErrorsEveryRebuild)  
  
    swApp.SetUserPreferenceToggle swShowErrorsEveryRebuild, False  
    Debug.Print "ShowErrorsEveryRebuild = " & swApp.GetUserPreferenceToggle(swShowErrorsEveryRebuild)  
  
End Sub
```

Suspend Automatic Rebuilds of an Assembly Example (VBA)

This example shows how to suspend automatic rebuilds of an assembly.

```
'-----  
' Preconditions:  
' 1. Verify that the specified assembly document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified assembly document.  
' 2. Disables automatic rebuilds of the assembly.  
' 3. Examine the Immediate window.  
'  
' NOTE: Because the assembly is used elsewhere, do not  
' save changes.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swAssemblyDoc As SldWorks.AssemblyDoc  
    Dim fileName As String  
    Dim errors As Long  
    Dim warnings As Long  
    Dim status As Boolean  
  
    Set swApp = Application.SldWorks  
  
    ' Open assembly  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\pillow_block.sldasm"  
    Set swAssemblyDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,  
warnings)  
  
    Debug.Print "State (False = rebuilds enabled; True = rebuilds suspended)"  
  
    ' Get current state  
    status = swAssemblyDoc.EnableAssemblyRebuild  
    Debug.Print "    Before running macro = " + CStr(swAssemblyDoc.EnableAssemblyRebuild)  
  
    ' Suspend automatic rebuilds of the assembly  
    swAssemblyDoc.EnableAssemblyRebuild = Not status  
    Debug.Print "    After running macro = " + CStr(swAssemblyDoc.EnableAssemblyRebuild)  
  
End Sub
```

Sweep Planar Loop Along Vector Example (VBA)

This example shows how to sweep a planar loop along a vector.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Select a planar face.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Sweeps a planar loop along the specified vector.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFace As SldWorks.Face2  
Dim swLoop As SldWorks.Loop2  
Dim vSweepOutput As Variant  
Dim swBody As SldWorks.Body2  
Dim swStopFace(1) As SldWorks.Face2  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swFace = swSelMgr.GetSelectedObject6(1, -1)  
Set swLoop = swFace.GetFirstLoop  
vSweepOutput = swLoop.SweepPlanarLoop(0#, 0#, 0.01, 0#)  
If vSweepOutput(0) Is Nothing Then  
    Debug.Print "Could not create temporary body; select a different face and rerun the macro."  
Else  
    Set swBody = vSweepOutput(0)  
    Set swStopFace(0) = vSweepOutput(1)  
    Set swStopFace(1) = vSweepOutput(2)  
  
    Debug.Print "Original face area      = " & swFace.GetArea * 1000000# & " mm^2"  
    Debug.Print "  Stop face 1 area      = " & swStopFace(0).GetArea * 1000000# & " mm^2"  
    Debug.Print "  Stop face 2 area      = " & swStopFace(1).GetArea * 1000000# & " mm^2"  
End If
```

End Sub

Switch Documents Example (C#)

This example shows how to switch documents by opening documents in their own and client model windows.

```
//-----
// Preconditions:
// 1. Verify that the specified documents to open exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified documents in their own
//    and client model windows.
// 2. Closes all open documents.
// 3. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FrameCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            Frame swFrame = default(Frame);
            ModelWindow swModelWindow = default(ModelWindow);
            object[] modelWindows = null;
            int errors = 0;
            int warnings = 0;
            int Hwnd = 0;
            string fileName = null;
            string strFolder = null;

            //Open the specified documents in their own windows
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\assemblymates\\knee.sldprt";
            swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "",
            ref errors, ref warnings);
            //Open client model window containing the active document
            swApp.CreateNewWindow();

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\assemblymates\\bracket.sldprt";
            swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "",
            ref errors, ref warnings);
            //Open client model window containing the active document
            swApp.CreateNewWindow();

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\assemblymates\\clamp.sldprt";
            swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "",
            ref errors, ref warnings);
            //Open client model window containing the active document
            swApp.CreateNewWindow();

            swFrame = (Frame)swApp.Frame();
            modelWindows = (object[])swFrame.ModelWindows;
            Debug.Print("Open documents in their own windows:");
            foreach (object obj in modelWindows)
            {
                swModelWindow = (ModelWindow)obj;
                //Get the model document in this model window
                swModelDoc = (ModelDoc2)swModelWindow.ModelDoc;
                //Rebuild the document
                swModelDoc.EditRebuild3();
                swModelDoc = null;
                //Show the model window
                Debug.Print("");
                swFrame.ShowModelWindow(swModelWindow);
                //Get and print the model window handle
                Hwnd = swModelWindow.Hwnd;
                Debug.Print(" Model window handle: " + Hwnd);
                //Get and print the model title as it is seen in the model window's title bar
                Debug.Print(" Model title as it seen in the model's window's title bar: " + swModelWindow.Title);
            }

            strFolder = "";
            //Specify true to close all documents, specify false to close
            //only the documents not modified
            swApp.CloseAllDocuments(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Switch Documents (VBA)

This example shows how to switch documents by opening documents in their own and client model windows.

```

'-----
' Preconditions:
' 1. Verify that the specified documents to open exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified documents in their own
'    and client model windows.
' 2. Click Window on the main menu to see a list of the
'    open documents.
' 3. Press F5 to continue.
' 4. Closes all open documents.
' 5. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swFrame As SldWorks.Frame
Dim swModelWindow As SldWorks.ModelWindow
Dim modelWindows As Variant
Dim obj As Variant
Dim errors As Long
Dim warnings As Long
Dim HWnd As Long
Dim fileName As String
Dim strFolder As String

Sub main()

    Set swApp = Application.SldWorks

    'Open the specified documents in their own windows
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\knee.sldprt"
    Set swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    'Open client model window containing the active document
    swApp.CreateNewWindow

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\bracket.sldprt"
    Set swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    'Open client model window containing the active document
    swApp.CreateNewWindow

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\clamp.sldprt"
    Set swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    'Open client model window containing the active document
    swApp.CreateNewWindow

    Set swFrame = swApp.Frame
    modelWindows = swFrame.modelWindows
    Debug.Print "Open documents in their own windows:"
    For Each obj In modelWindows
        Set swModelWindow = obj
        'Get the model document in this model window
        Set swModelDoc = swModelWindow.ModelDoc
        'Rebuild the document
        swModelDoc.EditRebuild3
        Set swModelDoc = Nothing
        'Show the model window
        Debug.Print ""
        swFrame.ShowModelWindow swModelWindow
        'Get and print the model window handle
        HWnd = swModelWindow.HWnd
        Debug.Print (" Model window handle: " & HWnd)
        'Get and print the model title as it is seen in the model window's title bar
        Debug.Print (" Model title as it seen in the model's window's title bar: " & swModelWindow.Title)
    Next obj

    Stop
    'Click Window on the main menu to see a list of the
    'open documents in their own windows
    'Press F5 to continue

    strFolder = ""
    'Specify true to close all documents, specify false to close
    'only the documents not modified
    swApp.CloseAllDocuments True

End Sub

```

Switch Documents Example (VB.NET)

This example shows how to switch documents by opening documents in their own and client model windows.

```

'-----
' Preconditions:
' 1. Verify that the specified documents to open exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified documents in their own
'    and client model windows.
' 2. Closes all open documents.
' 3. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModelDoc As ModelDoc2
        Dim swFrame As Frame
        Dim swModelWindow As ModelWindow
        Dim modelWindows() As Object
        Dim obj As Object
        Dim errors As Integer
        Dim warnings As Integer
        Dim Hwnd As Integer
        Dim fileName As String
        Dim strFolder As String

        'Open the specified documents in their own windows
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\knee.sldprt"
        swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        'Open client model window containing the active document
        swApp.CreateNewWindow()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\bracket.sldprt"
        swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        'Open client model window containing the active document
        swApp.CreateNewWindow()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\clamp.sldprt"
        swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        'Open client model window containing the active document
        swApp.CreateNewWindow()

        swFrame = swApp.Frame
        modelWindows = swFrame.ModelWindows
        Debug.Print("Open documents in their own windows:")
        For Each obj In modelWindows
            swModelWindow = obj
            'Get the model document in this model window
            swModelDoc = swModelWindow.ModelDoc
            'Rebuild the document
            swModelDoc.EditRebuild3()
            swModelDoc = Nothing
            'Show the model window
            Debug.Print("")
            swFrame.ShowModelWindow(swModelWindow)
            'Get and print the model window handle
            Hwnd = swModelWindow.Hwnd
            Debug.Print(" Model window handle: " & Hwnd)
            'Get and print the model title as it is seen in the model window's title bar
            Debug.Print(" Model title as it seen in the model's window's title bar: " & swModelWindow.Title)
        Next obj

        strFolder = ""
        'Specify true to close all documents, specify false to close
        'only the documents not modified
        swApp.CloseAllDocuments(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Switch Edit Context Example (C#)

This example shows how to switch back and forth between assembly and part when editing. It also shows how to fix and float an assembly component.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\assem2.sldasm
// 2. Press F5 after each macro pause.
//
// Postconditions: None
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace AssemblyDocMethods_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        AssemblyDoc assem;
        ModelDoc2 Part;

        bool boolstatus;

        public void Main()
        {
            assem = (AssemblyDoc)swApp.ActiveDoc;
            Part = (ModelDoc2)assem;

            boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            assem.FixComponent();
            // Part1^Assem2-1@assem2 is now fixed
            System.Diagnostics.Debugger.Break();

            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            assem.UnfixComponent();
            // Part1^Assem2-1@assem2 is now floating
            System.Diagnostics.Debugger.Break();

            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            int Info = 0;
            assem.EditPart2(true, true, ref Info);
            // Part1^Assem2-1@assem2 is being edited in the context of the assembly
            System.Diagnostics.Debugger.Break();

            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("assem2.SLDASM", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            assem.EditAssembly();
            // The assembly is being edited

        }

        public SldWorks swApp;
    }
}
```

Switch Edit Context Example (VBA)

This example shows how to switch back and forth between assembly and part when editing. It also shows how to fix and float an assembly component.

```

'-----
' Preconditions:
' 1. Open:
'   public_documents\samples\tutorial\api\assem2.sldasm
' 2. Press F5 after each macro pause.
'
' Postconditions: None
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
Dim swApp As SldWorks.SldWorks

Dim Part As SldWorks.AssemblyDoc
Dim boolstatus As Boolean
Option Explicit

Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Part.FixComponent
    ' Part1^Assem2-1@assem2 is now fixed
    Stop

    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Part.UnfixComponent
    ' Part1^Assem2-1@assem2 is now floating
    Stop

    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditPart2 True, True, 1
    ' Part1^Assem2-1@assem2 is being edited in the context of the assembly
    Stop

    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("assem2.SLDASM", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditAssembly
    ' The assembly is being edited

End Sub

```

Switch Edit Context Example (VB.NET)

This example shows how to switch back and forth between assembly and part when editing. It also shows how to fix and float an assembly component.

```

'-----
' Preconditions:
' 1. Open:
'   public_documents\samples\tutorial\api\assem2.sldasm
' 2. Press F5 after each macro pause.
'
' Postconditions: None
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim assem As AssemblyDoc
    Dim Part As ModelDoc2
    Dim boolstatus As Boolean

    Sub main()

        assem = swApp.ActiveDoc
        Part = assem

        boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        assem.FixComponent()
        ' Part1^Assem2-1@assem2 is now fixed
        Stop

        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        assem.UnfixComponent()
        ' Part1^Assem2-1@assem2 is now floating
        Stop

        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Part1^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        assem.EditPart2(True, True, 1)
        ' Part1^Assem2-1@assem2 is being edited in the context of the assembly
        Stop

        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("assem2.SLDASM", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        assem.EditAssembly()
        ' The assembly is being edited

    End Sub

    Public swApp As SldWorks

End Class

```